

Tarea2_Contreras_Zuñiga

June 5, 2026

1 Tarea 2: Análisis de panel

Alumno: Álvaro Contreras Zúñiga

Matricula: 2021433031

```
[1]: #Importación librerías

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import sklearn
import scipy
from scipy.stats import nbinom
import seaborn as sns
from statsmodels.iolib.summary2 import summary_col
from sklearn.preprocessing import StandardScaler
import datetime
import random
import linearmodels.panel as lmp
from linearmodels.iv import IV2SLS
from scipy import stats
import numpy.linalg as la
from linearmodels.panel import PanelOLS

import warnings
warnings.filterwarnings("ignore")

%matplotlib inline
```

Diccionario - “iso_code”: “Country or region code”, - “date”: “Date of the record”,
- “retail_and_recreation_percent_change_from_baseline”: “Change in retail/recreation activity”,
- “grocery_and_pharmacy_percent_change_from_baseline”: “Change in grocery/pharmacy activity”,
- “parks_percent_change_from_baseline”: “Change in park visits”,
- “transit_stations_percent_change_from_baseline”: “Change in transit station activity”,
- “workplaces_raw”: “Raw workplace activity data”,
- “residential_percent_change_from_baseline”: “Change in residential activity”,
- “trend”: “Trend

indicator”, - “workplaces”: “Processed workplace activity data”, - “Valor_Stringency_Index”: “Stringency index value”, - “Valor_GovernmentResponseIndex”: “Government response index value”, - “Valor_EconomicSupportIndex”: “Economic support index value”, - “Valor_Containment_Health_index”: “Containment and health index value”, - “workplace_closing”: “Workplace closing indicator”, - “daily_cases”: “Daily reported cases”, - “week”: “Week number”, - “year”: “Year”, - “CODE”: “Region code”, - “NAME”: “Region name”, - “Population”: “Population of the region”, - “agriculture”: “Agriculture sector data”, - “industry”: “Industry sector data”, - “construction”: “Construction sector data”, - “age_dependency”: “Age dependency ratio”, - “old_age_dependency”: “Old age dependency ratio”, - “young_age_dependency”: “Young age dependency ratio”, - “sex_ratio”: “Sex ratio”, - “unemp”: “Unemployment rate”, - “f_unemp”: “Female unemployment rate”, - “m_unemp”: “Male unemployment rate”, - “foreigners”: “Foreign population percentage”, - “country”: “Country name”

Variable	Descripción Original	Definición en el Contexto del Modelo
iso_code	Country or region code	Código estándar ISO de la región o país.
date	Date of the record	Fecha exacta de la observación. Actúa como la coordenada temporal del panel.
retail_and_recreation_percent_change_from_baseline	Change in retail/leisure activity	Variación porcentual en visitas a restaurantes, centros comerciales, museos y cines respecto a la línea base pre-pandemia.
grocery_and_pharmacy_percent_change_from_baseline	Change in grocery/pharmacy activity	Variación porcentual en visitas a supermercados y farmacias respecto a la línea base.
parks_percent_change_from_baseline	Change in park visits	Variación porcentual en visitas a parques respecto a la línea base.
transit_stations_percent_change_from_baseline	Change in public transportation activity	Variación porcentual en el uso de estaciones de transporte público (metro, buses, trenes).
workplaces_raw	Raw workplace activity data	Datos crudos (sin suavizado matemático) de la variación de movilidad hacia lugares de trabajo.
residential_percent_change_from_baseline	Change in residential activity	Variación porcentual en el tiempo de permanencia en zonas residenciales (hogares).
trend	Trend indicator	Variable matemática o estadística que captura la tendencia temporal subyacente de la serie.

Variable	Descripción Original	Definición en el Contexto del Modelo
workplaces	Processed workplace activity data	Variación porcentual procesada en la movilidad hacia lugares de trabajo respecto a la línea base.
Valor_Stringency_Index	Stringency index value	Índice de Rigurosidad (0-100). Mide la severidad de las políticas de cierre gubernamentales.
Valor_GovernmentResponseIndex	Government response index value	Índice de Respuesta Gubernamental (0-100). Combina restricciones de movilidad, medidas de salud y apoyo económico.
Valor_EconomicSupportIndex	Economic support index value	Índice de Apoyo Económico (0-100). Mide las medidas de ayuda financiera estatal a los ciudadanos (ej. subsidios, alivio de deudas).
Valor_Containment_Health_index	Containment and health index value	Índice de Contención y Salud (0-100). Combina restricciones de movilidad con medidas de salud pública (testeo, mascarillas).
workplace_closing	Workplace closing indicator	Nivel de severidad de la política legal respecto al cierre obligatorio de lugares de trabajo.
daily_cases	Daily reported cases	Número de casos nuevos de COVID-19 reportados diariamente.
week	Week number	Contador numérico de la semana dentro de la muestra temporal del panel.
year	Year	Año cronológico de la observación (2020, 2021, etc.).
CODE	Region code	Código interno único para identificar a cada ciudad/región. Actúa como el identificador de entidad (Individuo <i>i</i>) del panel.
NAME	Region name	Nombre geográfico de la ciudad o región analizada.
Population	Population of the region	Población total de la ciudad o región.

Variable	Descripción Original	Definición en el Contexto del Modelo
agriculture	Agriculture sector data	Proporción del sector agrícola dentro de la matriz productiva de la región.
industry	Industry sector data	Proporción del sector industrial dentro de la matriz productiva de la región.
construction	Construction sector data	Proporción del sector de la construcción dentro de la matriz productiva de la región.
age_dependency	Age dependency ratio	Tasa de dependencia demográfica global. Mide la proporción de la población económicamente inactiva (jóvenes y ancianos) respecto a la fuerza laboral activa.
old_age_dependency	Old age dependency ratio	Tasa de dependencia demográfica exclusiva de la tercera edad.
young_age_dependency	Young age dependency ratio	Tasa de dependencia demográfica exclusiva de jóvenes e infantes.
sex_ratio	Sex ratio	Razón de sexo (proporción demográfica entre hombres y mujeres en el territorio).
unemp	Unemployment rate	Tasa global de desempleo de la región.
f_unemp	Female unemployment rate	Tasa específica de desempleo femenino de la región.
m_unemp	Male unemployment rate	Tasa específica de desempleo masculino de la región.
foreigners	Foreign population percentage	Porcentaje de población residente de origen extranjero o inmigrante.
country	Country name	Identificador nominal (texto) del país al que pertenece la observación regional.

Pregunta 1

La base de datos de este estudio contiene información de varias ciudades durante la época del COVID-19. En esta base se registra cosas como la cantidad de casos diarios, las restricciones del gobierno (cuarentenas, cierres de comercios) y variables de la población (tasa de desempleo, porcentaje de extranjeros, etc.).

Objetivo del estudio: El objetivo es evaluar el impacto de las restricciones gubernamentales sobre

la movilidad laboral durante la pandemia, se quiere la pregunta ¿En qué medida las restricciones sanitarias y de confinamiento dictadas por los gobiernos locales redujeron la movilidad de las personas hacia sus lugares de trabajo?

Hipótesis principal: Existe una relación inversa y estadísticamente significativa entre el nivel de rigurosidad de las medidas gubernamentales (Valor_Stringency_Index o workplace_closing) y la variación de la movilidad laboral (workplaces), se espera que a mayor severidad de las restricciones, se produzca una disminución en la asistencia presencial a los puestos de trabajo.

En primer lugar se carga la data original, se solicita el número de filas y columnas para tener un registro de la cantidad de datos que se tienen. Luego de esto, se eliminan las columnas que no se encuentran como variables en el diccionario ya que no se tiene información sobre estas. Por último se obtiene la informacion general utilizando df.info.

```
[2]: # Cargar base de datos original
df_inicial = pd.read_csv('dataset_prueba.csv', delimiter=",", decimal='.')
print(f"Dimensiones originales: {df_inicial.shape[0]} filas, {df_inicial.
↳shape[1]} columnas\n")

# Eliminación de columnas que no estan en el diccionario
columnas_a_borrar = ['d1', 'd2', 'd3', 'd4', 'd5', 'd6', 'edu2', 'edu3']
df = df_inicial.drop(columns=columnas_a_borrar)
print(f"Dimensiones tras borrar variables fuera del diccionario: {df.shape[0]}_
↳filas, {df.shape[1]} columnas\n")

# Informacion general actual y tipos de datos

df.info()
```

Dimensiones originales: 12040 filas, 41 columnas

Dimensiones tras borrar variables fuera del diccionario: 12040 filas, 33 columnas

```
<class 'pandas.DataFrame'>
RangeIndex: 12040 entries, 0 to 12039
Data columns (total 33 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   iso_code                                  12040 non-null  str
1   date                                      12040 non-null  str
2   retail_and_recreation_percent_change_from_baseline  12040 non-null  float64
3   grocery_and_pharmacy_percent_change_from_baseline  12040 non-null  float64
4   parks_percent_change_from_baseline              12040 non-null  float64
5   transit_stations_percent_change_from_baseline      12040 non-null  float64
6   workplaces_raw                                12040 non-null  float64
7   residential_percent_change_from_baseline          12040 non-null  float64
8   trend                                           12040 non-null  float64
9   workplaces                                     12040 non-null  float64
```

```

10 Valor_Stringency_Index          12040 non-null float64
11 Valor_GovernmentResponseIndex    12040 non-null float64
12 Valor_EconomicSupportIndex       12040 non-null float64
13 Valor_Containment_Health_index   12040 non-null float64
14 workplace_closing               12040 non-null float64
15 daily_cases                     12040 non-null float64
16 week                           12040 non-null int64
17 year                           12040 non-null int64
18 CODE                           12040 non-null str
19 NAME                           12040 non-null str
20 Population                      12040 non-null float64
21 agriculture                     12040 non-null float64
22 industry                        12040 non-null float64
23 construction                    12040 non-null float64
24 age_dependency                  12040 non-null float64
25 old_age_dependency              12040 non-null float64
26 young_age_dependency            12040 non-null float64
27 sex_ratio                       12040 non-null float64
28 unemp                          12040 non-null float64
29 f_unemp                        12040 non-null float64
30 m_unemp                        12040 non-null float64
31 foreigners                      12040 non-null float64
32 country                         12040 non-null str
dtypes: float64(26), int64(2), str(5)
memory usage: 3.0 MB

```

```
[3]: pd.set_option('display.max_columns', None)
```

```
df
```

```

[3]:      iso_code      date \
0      DE-BE  2020-02-17
1      DE-BE  2020-02-24
2      DE-BE  2020-03-02
3      DE-BE  2020-03-09
4      DE-BE  2020-03-16
...
12035    SE-M  2022-09-19
12036    SE-M  2022-09-26
12037    SE-M  2022-10-03
12038    SE-M  2022-10-10
12039    SE-M  2022-10-17

      retail_and_recreation_percent_change_from_baseline \
0                                     6.0
1                                     1.6
2                                     2.6

```

3	-3.4
4	-11.8
...	...
12035	-3.4
12036	-0.2
12037	-1.2
12038	-3.2
12039	-5.5

	grocery_and_pharmacy_percent_change_from_baseline \
0	3.00
1	0.00
2	6.40
3	3.40
4	11.40
...	...
12035	5.20
12036	7.40
12037	6.00
12038	4.20
12039	2.75

	parks_percent_change_from_baseline \
0	15.00
1	8.60
2	8.20
3	-1.00
4	-4.60
...	...
12035	50.60
12036	62.60
12037	31.80
12038	43.80
12039	39.75

	transit_stations_percent_change_from_baseline	workplaces_raw \
0	4.0	1.00
1	0.0	0.20
2	-0.4	0.40
3	-5.8	-0.60
4	-16.0	-7.80
...	...	...
12035	-16.0	-12.80
12036	-16.8	-12.00
12037	-17.8	-10.80
12038	-16.8	-8.80
12039	-17.0	-8.25

	residential_percent_change_from_baseline	trend	workplaces \
0	0.00	9.266668	4.244073
1	0.60	5.006175	2.468286
2	1.00	-3.765428	-2.845394
3	1.80	-12.537031	-11.510297
4	4.60	-21.308634	-21.800890
...	...	...	...
12035	2.20	-15.047446	-8.365623
12036	1.80	-15.175740	-6.167200
12037	2.40	-15.304033	-5.122127
12038	2.20	-15.432327	-5.921463
12039	2.25	-15.544126	-7.332893

	Valor_Stringency_Index	Valor_GovernmentResponseIndex \
0	0.00	0.00
1	0.00	0.00
2	0.00	0.00
3	0.00	0.00
4	0.00	0.00
...	...	...
12035	11.11	16.67
12036	11.11	14.58
12037	11.11	14.58
12038	11.11	14.58
12039	11.11	14.58

	Valor_EconomicSupportIndex	Valor_Containment_Health_index \
0	0.0	0.00
1	0.0	0.00
2	0.0	0.00
3	0.0	0.00
4	0.0	0.00
...	...	...
12035	0.0	19.05
12036	0.0	16.67
12037	0.0	16.67
12038	0.0	16.67
12039	0.0	16.67

	workplace_closing	daily_cases	week	year	CODE	NAME	Population \
0	0.0	0.0160	1	2020	DE001C	Berlin	3669.491
1	0.0	0.0160	2	2020	DE001C	Berlin	3669.491
2	0.0	0.0594	3	2020	DE001C	Berlin	3669.491
3	0.0	0.5572	4	2020	DE001C	Berlin	3669.491
4	0.0	3.2780	5	2020	DE001C	Berlin	3669.491
...	...	...	...	...	...	...	...

12035	0.0	0.0000	136	2022	SE003C	Malmö	333.633
12036	0.0	0.0000	137	2022	SE003C	Malmö	333.633
12037	0.0	0.0000	138	2022	SE003C	Malmö	333.633
12038	0.0	0.0000	139	2022	SE003C	Malmö	333.633
12039	0.0	0.0000	140	2022	SE003C	Malmö	333.633

	agriculture	industry	construction	age_dependency	\
0	0.000419	0.076745	0.046983	0.592713	
1	0.000419	0.076745	0.046983	0.592713	
2	0.000419	0.076745	0.046983	0.592713	
3	0.000419	0.076745	0.046983	0.592713	
4	0.000419	0.076745	0.046983	0.592713	
...	...	...	...	...	
12035	0.001360	0.070238	0.061295	0.617317	
12036	0.001360	0.070238	0.061295	0.617317	
12037	0.001360	0.070238	0.061295	0.617317	
12038	0.001360	0.070238	0.061295	0.617317	
12039	0.001360	0.070238	0.061295	0.617317	

	old_age_dependency	young_age_dependency	sex_ratio	unemp	f_unemp	\
0	0.290913	0.306017	103.377815	8.0	0.086068	
1	0.290913	0.306017	103.377815	8.0	0.086068	
2	0.290913	0.306017	103.377815	8.0	0.086068	
3	0.290913	0.306017	103.377815	8.0	0.086068	
4	0.290913	0.306017	103.377815	8.0	0.086068	
...	...	...	...	...	...	
12035	0.370739	0.246578	102.195691	15.0	0.165522	
12036	0.370739	0.246578	102.195691	15.0	0.165522	
12037	0.370739	0.246578	102.195691	15.0	0.165522	
12038	0.370739	0.246578	102.195691	15.0	0.165522	
12039	0.370739	0.246578	102.195691	15.0	0.165522	

	m_unemp	foreigners	country
0	0.071170	9.0	DE
1	0.071170	9.0	DE
2	0.071170	9.0	DE
3	0.071170	9.0	DE
4	0.071170	9.0	DE
...	...	...	...
12035	0.134529	8.0	SE
12036	0.134529	8.0	SE
12037	0.134529	8.0	SE
12038	0.134529	8.0	SE
12039	0.134529	8.0	SE

[12040 rows x 33 columns]

Para revisar las variables se ejecuta este script para observar su cardinalidad

[4]: *#Diagnóstico de Valores Únicos*

```
print("--- Diagnóstico de Valores Únicos (Variables del Modelo) ---")
for col in df.columns:
    valores_unicos = df[col].nunique()
    tipo_dato = df[col].dtype

    if valores_unicos < 50:
        print(f"[{tipo_dato}] '{col}': {valores_unicos} valores -> {df[col].
↪unique()}")
    else:
        print(f"[{tipo_dato}] '{col}': {valores_unicos} valores (Alta_
↪cardinalidad)")
```

```
--- Diagnóstico de Valores Únicos (Variables del Modelo) ---
[str] 'iso_code': 54 valores (Alta cardinalidad)
[str] 'date': 140 valores (Alta cardinalidad)
[float64] 'retail_and_recreation_percent_change_from_baseline': 695 valores
(Alta cardinalidad)
[float64] 'grocery_and_pharmacy_percent_change_from_baseline': 586 valores (Alta
cardinalidad)
[float64] 'parks_percent_change_from_baseline': 1580 valores (Alta cardinalidad)
[float64] 'transit_stations_percent_change_from_baseline': 803 valores (Alta
cardinalidad)
[float64] 'workplaces_raw': 457 valores (Alta cardinalidad)
[float64] 'residential_percent_change_from_baseline': 220 valores (Alta
cardinalidad)
[float64] 'trend': 7560 valores (Alta cardinalidad)
[float64] 'workplaces': 7560 valores (Alta cardinalidad)
[float64] 'Valor_Stringency_Index': 321 valores (Alta cardinalidad)
[float64] 'Valor_GovernmentResponseIndex': 362 valores (Alta cardinalidad)
[float64] 'Valor_EconomicSupportIndex': 23 valores -> [ 0.   55.   62.5  87.5
85.   77.5  67.5  37.5  22.5  20.  100.   75.
 70.   50.   30.   25.   42.5  65.   57.5  52.5  47.5  12.5   7.5]
[float64] 'Valor_Containment_Health_index': 355 valores (Alta cardinalidad)
[float64] 'workplace_closing': 16 valores -> [0.   0.2  1.2  2.   2.2  3.   2.6  1.8  1.
1.4  2.8  2.4  1.6  0.4  0.8  0.6]
[float64] 'daily_cases': 532 valores (Alta cardinalidad)
[int64] 'week': 140 valores (Alta cardinalidad)
[int64] 'year': 3 valores -> [2020 2021 2022]
[str] 'CODE': 86 valores (Alta cardinalidad)
[str] 'NAME': 86 valores (Alta cardinalidad)
[float64] 'Population': 185 valores (Alta cardinalidad)
[float64] 'agriculture': 73 valores (Alta cardinalidad)
[float64] 'industry': 86 valores (Alta cardinalidad)
[float64] 'construction': 86 valores (Alta cardinalidad)
[float64] 'age_dependency': 86 valores (Alta cardinalidad)
```

```
[float64] 'old_age_dependency': 86 valores (Alta cardinalidad)
[float64] 'young_age_dependency': 86 valores (Alta cardinalidad)
[float64] 'sex_ratio': 185 valores (Alta cardinalidad)
[float64] 'unemp': 26 valores -> [ 8.  4.  5.  6.  7.  9.  3. 21. 25. 15. 32.
16. 36. 34. 28. 17. 24. 19.
12. 20. 14. 18. 13. 10. 26. 27.]
[float64] 'f_unemp': 86 valores (Alta cardinalidad)
[float64] 'm_unemp': 86 valores (Alta cardinalidad)
[float64] 'foreigners': 14 valores -> [ 9. 11. 10. 12. 15.  3.  6.  7.  8.  5.
4. 13. 14.  2.]
[str] 'country': 5 valores -> <StringArray>
['DE', 'ES', 'FR', 'IT', 'SE']
Length: 5, dtype: str
```

Se realiza un conteo de datos faltantes

```
[5]: # Diagnóstico de Datos Faltantes
print("\n--- Conteo de Valores Nulos ---")
nulos = df.isnull().sum()
print(nulos[nulos > 0].sort_values(ascending=False))

print(df.isnull().sum())
```

```
--- Conteo de Valores Nulos ---
Series([], dtype: int64)
iso_code                                0
date                                    0
retail_and_recreation_percent_change_from_baseline  0
grocery_and_pharmacy_percent_change_from_baseline  0
parks_percent_change_from_baseline          0
transit_stations_percent_change_from_baseline  0
workplaces_raw                             0
residential_percent_change_from_baseline      0
trend                                       0
workplaces                                 0
Valor_Stringency_Index                    0
Valor_GovernmentResponseIndex              0
Valor_EconomicSupportIndex                 0
Valor_Containment_Health_index             0
workplace_closing                         0
daily_cases                               0
week                                       0
year                                       0
CODE                                       0
NAME                                       0
Population                                0
agriculture                               0
industry                                  0
```

```

construction          0
age_dependency         0
old_age_dependency     0
young_age_dependency   0
sex_ratio              0
unemp                 0
f_unemp               0
m_unemp               0
foreigners            0
country               0
dtype: int64

```

Se realiza un nuevo control de la información

```

[6]: # Control de pérdida de información

dim_final = df.shape
print(f"--- Dimensiones Finales tras Limpieza e Indexación ---")
print(f"Observaciones retenidas: {dim_final[0]} | Variables finales: {dim_final[1]}")
print(f"Pérdida de filas por atrición: {df_inicial.shape[0] - dim_final[0]}\n")

```

```

--- Dimensiones Finales tras Limpieza e Indexación ---
Observaciones retenidas: 12040 | Variables finales: 33
Pérdida de filas por atrición: 0

```

Se convierte la fecha a datetime

```

[7]: df['date'] = pd.to_datetime(df['date'])

```

Se hace una revision de cada columna

```

[8]: df.describe()

```

```

[8]:
count      date \
count      12040
mean  2021-06-17 12:00:00
min    2020-02-17 00:00:00
25%    2020-10-17 06:00:00
50%    2021-06-17 12:00:00
75%    2022-02-15 18:00:00
max    2022-10-17 00:00:00
std      NaN

count  retail_and_recreation_percent_change_from_baseline \
count      12040.000000
mean      -16.437272
min      -95.600000

```

25%	-26.400000
50%	-11.400000
75%	-3.600000
max	74.600000
std	21.599436

	grocery_and_pharmacy_percent_change_from_baseline \
count	12040.000000
mean	5.124340
min	-66.200000
25%	-3.200000
50%	6.200000
75%	15.000000
max	83.600000
std	15.707311

	parks_percent_change_from_baseline \
count	12040.000000
mean	49.675619
min	-95.800000
25%	4.800000
50%	34.200000
75%	78.800000
max	631.000000
std	71.697845

	transit_stations_percent_change_from_baseline	workplaces_raw \
count	12040.000000	12040.000000
mean	-17.602313	-25.002928
min	-88.600000	-87.000000
25%	-31.800000	-32.400000
50%	-18.400000	-22.800000
75%	-4.400000	-14.400000
max	123.200000	6.600000
std	24.540594	14.681099

	residential_percent_change_from_baseline	trend	workplaces \
count	12040.000000	12040.000000	12040.000000
mean	7.071179	-19.004247	-18.667353
min	-3.800000	-80.071581	-76.787132
25%	3.200000	-22.614732	-24.358379
50%	5.600000	-17.430940	-16.819961
75%	9.400000	-13.324697	-10.883994
max	42.600000	12.355647	19.057630
std	6.022794	9.857500	11.801356

Valor_Stringency_Index	Valor_GovernmentResponseIndex \
------------------------	---------------------------------

count	12040.000000	12040.000000
mean	27.486254	30.636789
min	0.000000	0.000000
25%	0.000000	0.000000
50%	15.858000	32.030000
75%	53.420000	60.160000
max	93.520000	81.306000
std	29.835937	30.244900

	Valor_EconomicSupportIndex	Valor_Containment_Health_index \
count	12040.000000	12040.000000
mean	30.955150	30.591686
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	36.610000
75%	75.000000	59.230000
max	100.000000	85.420000
std	36.364281	29.943118

	workplace_closing	daily_cases	week	year \
count	12040.000000	12040.000000	12040.000000	12040.000000
mean	0.850831	2355.733422	70.500000	2020.971429
min	0.000000	0.000000	1.000000	2020.000000
25%	0.000000	0.037200	35.750000	2020.000000
50%	0.000000	632.010400	70.500000	2021.000000
75%	2.000000	3815.644400	105.250000	2022.000000
max	3.000000	22203.314200	140.000000	2022.000000
std	0.997067	3154.914356	40.415166	0.792343

	Population	agriculture	industry	construction	age_dependency \
count	12040.000000	12040.000000	12040.000000	12040.000000	12040.000000
mean	745.474163	0.006719	0.115376	0.053282	0.653934
min	201.048000	0.000419	0.036570	0.018195	0.520499
25%	246.794000	0.001350	0.081304	0.037263	0.609178
50%	346.790500	0.003547	0.107078	0.052972	0.650998
75%	620.523000	0.010854	0.148541	0.069632	0.681714
max	10274.884000	0.071369	0.232640	0.152669	0.837478
std	1277.341567	0.009605	0.044462	0.018237	0.072849

	old_age_dependency	young_age_dependency	sex_ratio	unemp \
count	12040.000000	12040.000000	12040.000000	12040.000000
mean	0.320896	0.334465	106.714479	12.220515
min	0.255039	0.238051	91.448785	3.000000
25%	0.280293	0.282108	103.522992	6.000000
50%	0.306745	0.331407	107.013220	8.000000
75%	0.338193	0.366313	109.531509	18.000000
max	0.485328	0.507336	115.752855	36.000000

std	0.052749	0.062533	4.247242	8.265315
-----	----------	----------	----------	----------

	f_unemp	m_unemp	foreigners
count	12040.000000	12040.000000	12040.000000
mean	0.113865	0.121292	8.145515
min	0.035794	0.024618	2.000000
25%	0.061770	0.052677	6.000000
50%	0.085044	0.078852	9.000000
75%	0.149918	0.163352	11.000000
max	0.328653	0.427472	15.000000
std	0.070448	0.093873	2.991933

```
[9]: print("--- Revisión de columna por columna ---")

for col in df.columns:
    tipo = df[col].dtype
    print(f"\n * Variable: '{col}' | Tipo: {tipo}")

    # Si la columna es un número
    if pd.api.types.is_numeric_dtype(df[col]):
        minimo = df[col].min()
        maximo = df[col].max()
        media = df[col].mean()

        print(f"    Rango de valores : De {minimo:.2f} a {maximo:.2f}")
        print(f"    Promedio (Media) : {media:.2f}")

        # Alerta visual para detectar posibles anomalías rápidamente
        if minimo < 0:
            print("    ALERTA: Contiene valores negativos. (Verificar si tiene
↳ sentido lógico).")

    # Si la columna es texto o fecha
    else:
        unicos = df[col].nunique()
        print(f"    Categorías únicas: {unicos}")
        # Muestra los 5 valores que más se repiten para detectar errores de
↳ tipeo
        print(f"    Valores más comunes:\n{df[col].value_counts().head(5).
↳ to_string()}")
```

--- Revisión de columna por columna ---

```
* Variable: 'iso_code' | Tipo: str
Categorías únicas: 54
Valores más comunes:
iso_code
DE-NW    2100
```

DE-BW 560
DE-BY 420
DE-HE 420
DE-SN 420

* Variable: 'date' | Tipo: datetime64[us]

Categorías únicas: 140

Valores más comunes:

date

2020-02-17 86
2020-02-24 86
2020-03-02 86
2020-03-09 86
2020-03-16 86

* Variable: 'retail_and_recreation_percent_change_from_baseline' | Tipo: float64

Rango de valores : De -95.60 a 74.60

Promedio (Media) : -16.44

ALERTA: Contiene valores negativos. (Verificar si tiene sentido lógico).

* Variable: 'grocery_and_pharmacy_percent_change_from_baseline' | Tipo: float64

Rango de valores : De -66.20 a 83.60

Promedio (Media) : 5.12

ALERTA: Contiene valores negativos. (Verificar si tiene sentido lógico).

* Variable: 'parks_percent_change_from_baseline' | Tipo: float64

Rango de valores : De -95.80 a 631.00

Promedio (Media) : 49.68

ALERTA: Contiene valores negativos. (Verificar si tiene sentido lógico).

* Variable: 'transit_stations_percent_change_from_baseline' | Tipo: float64

Rango de valores : De -88.60 a 123.20

Promedio (Media) : -17.60

ALERTA: Contiene valores negativos. (Verificar si tiene sentido lógico).

* Variable: 'workplaces_raw' | Tipo: float64

Rango de valores : De -87.00 a 6.60

Promedio (Media) : -25.00

ALERTA: Contiene valores negativos. (Verificar si tiene sentido lógico).

* Variable: 'residential_percent_change_from_baseline' | Tipo: float64

Rango de valores : De -3.80 a 42.60

Promedio (Media) : 7.07

ALERTA: Contiene valores negativos. (Verificar si tiene sentido lógico).

* Variable: 'trend' | Tipo: float64

Rango de valores : De -80.07 a 12.36

Promedio (Media) : -19.00

ALERTA: Contiene valores negativos. (Verificar si tiene sentido lógico).

* Variable: 'workplaces' | Tipo: float64

Rango de valores : De -76.79 a 19.06

Promedio (Media) : -18.67

ALERTA: Contiene valores negativos. (Verificar si tiene sentido lógico).

* Variable: 'Valor_Stringency_Index' | Tipo: float64

Rango de valores : De 0.00 a 93.52

Promedio (Media) : 27.49

* Variable: 'Valor_GovernmentResponseIndex' | Tipo: float64

Rango de valores : De 0.00 a 81.31

Promedio (Media) : 30.64

* Variable: 'Valor_EconomicSupportIndex' | Tipo: float64

Rango de valores : De 0.00 a 100.00

Promedio (Media) : 30.96

* Variable: 'Valor_Containment_Health_index' | Tipo: float64

Rango de valores : De 0.00 a 85.42

Promedio (Media) : 30.59

* Variable: 'workplace_closing' | Tipo: float64

Rango de valores : De 0.00 a 3.00

Promedio (Media) : 0.85

* Variable: 'daily_cases' | Tipo: float64

Rango de valores : De 0.00 a 22203.31

Promedio (Media) : 2355.73

* Variable: 'week' | Tipo: int64

Rango de valores : De 1.00 a 140.00

Promedio (Media) : 70.50

* Variable: 'year' | Tipo: int64

Rango de valores : De 2020.00 a 2022.00

Promedio (Media) : 2020.97

* Variable: 'CODE' | Tipo: str

Categorías únicas: 86

Valores más comunes:

CODE

DE001C 140

DE007C 140

DE025C 140

DE033C 140

DE081C 140

* Variable: 'NAME' | Tipo: str
Categorías únicas: 86
Valores más comunes:

NAME

Berlin	140
Stuttgart	140
Freiburg im Breisgau	140
Karlsruhe	140
Mannheim	140

* Variable: 'Population' | Tipo: float64
Rango de valores : De 201.05 a 10274.88
Promedio (Media) : 745.47

* Variable: 'agriculture' | Tipo: float64
Rango de valores : De 0.00 a 0.07
Promedio (Media) : 0.01

* Variable: 'industry' | Tipo: float64
Rango de valores : De 0.04 a 0.23
Promedio (Media) : 0.12

* Variable: 'construction' | Tipo: float64
Rango de valores : De 0.02 a 0.15
Promedio (Media) : 0.05

* Variable: 'age_dependency' | Tipo: float64
Rango de valores : De 0.52 a 0.84
Promedio (Media) : 0.65

* Variable: 'old_age_dependency' | Tipo: float64
Rango de valores : De 0.26 a 0.49
Promedio (Media) : 0.32

* Variable: 'young_age_dependency' | Tipo: float64
Rango de valores : De 0.24 a 0.51
Promedio (Media) : 0.33

* Variable: 'sex_ratio' | Tipo: float64
Rango de valores : De 91.45 a 115.75
Promedio (Media) : 106.71

* Variable: 'unemp' | Tipo: float64
Rango de valores : De 3.00 a 36.00
Promedio (Media) : 12.22

```

* Variable: 'f_unemp' | Tipo: float64
  Rango de valores : De 0.04 a 0.33
  Promedio (Media) : 0.11

* Variable: 'm_unemp' | Tipo: float64
  Rango de valores : De 0.02 a 0.43
  Promedio (Media) : 0.12

* Variable: 'foreigners' | Tipo: float64
  Rango de valores : De 2.00 a 15.00
  Promedio (Media) : 8.15

* Variable: 'country' | Tipo: str
  Categorías únicas: 5
  Valores más comunes:
country
DE      5460
ES      2240
FR      2100
IT      1960
SE       280

```

```

[10]: import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

# Configuración de estilo visual premium
sns.set_style("darkgrid")
sns.set_context("notebook", font_scale=0.85)

# Obtener lista de columnas numericas usando exclusion
columnas_num = df.select_dtypes(exclude=['object', 'datetime']).columns
total_columnas = len(columnas_num)

# Calcular disposicion de los graficos (5 columnas)
cols_grafico = 5
filas_grafico = (total_columnas + cols_grafico - 1) // cols_grafico

# Parte 1: Graficos de Distribucion (Histogramas + Curvas de Densidad)
fig_hist, ejes_hist = plt.subplots(filas_grafico, cols_grafico, figsize=(22, 3.
    ↪ 5 * filas_grafico))
ejes_planos_hist = ejes_hist.flatten()

print(f"Distribuciones para {total_columnas} variables...")

for eje, nombre_col in zip(ejes_planos_hist, columnas_num):

```

```

sns.histplot(
    data=df,
    x=nombre_col,
    kde=True,
    ax=eje,
    color='teal',
    alpha=0.5,
    linewidth=0.5
)
eje.set_title(nombre_col, fontsize=10, fontweight='bold')
eje.set_xlabel('')
eje.set_ylabel('')

# Ocultar paneles sobrantes en la grilla
for idx in range(total_columnas, len(ejes_planos_hist)):
    fig_hist.delaxes(ejes_planos_hist[idx])

plt.tight_layout()
plt.show()

# Parte 2: Boxplots para Deteccion de Valores Atipicos
fig_box, ejes_box = plt.subplots(filas_grafico, cols_grafico, figsize=(22, 2.5*
    ↪ filas_grafico))
ejes_planos_box = ejes_box.flatten()

print(f"Boxplots para {total_columnas} variables...")

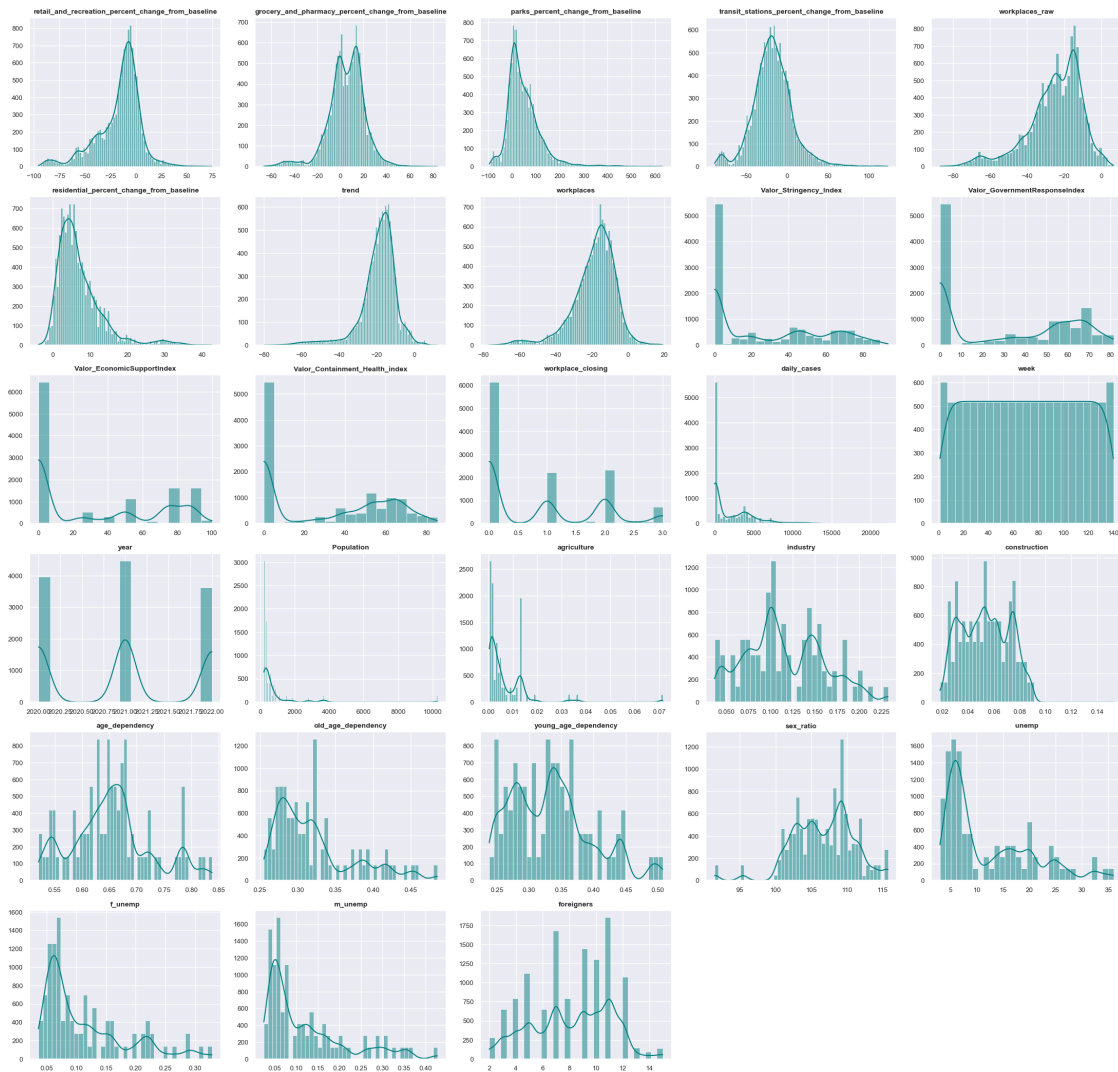
for eje, nombre_col in zip(ejes_planos_box, columnas_num):
    sns.boxplot(
        data=df,
        x=nombre_col,
        ax=eje,
        color='mediumpurple',
        fliersize=3,
        flierprops={"marker": "o", "markerfacecolor": "darkorange",
    ↪ "markeredgecolor": "darkorange"}
    )
    eje.set_title(nombre_col, fontsize=10, fontweight='bold')
    eje.set_xlabel('')

# Ocultar paneles sobrantes en la grilla
for idx in range(total_columnas, len(ejes_planos_box)):
    fig_box.delaxes(ejes_planos_box[idx])

plt.tight_layout()
plt.show()

```

Distribuciones para 28 variables...



Boxplots para 28 variables...

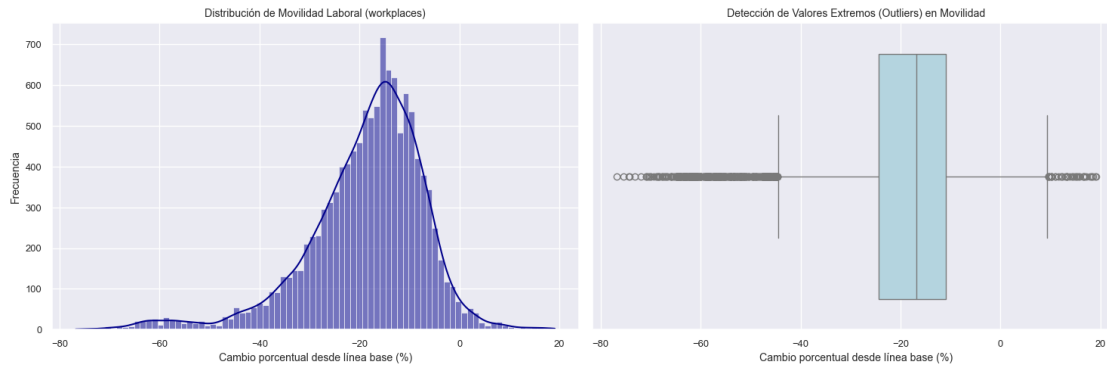


```
[11]: # Diagnóstico Visual de la Variable Dependiente (Workplaces)
fig, ax = plt.subplots(1, 2, figsize=(15, 5))

# Histograma
sns.histplot(df['workplaces'], kde=True, color='darkblue', ax=ax[0])
ax[0].set_title('Distribución de Movilidad Laboral (workplaces)')
ax[0].set_xlabel('Cambio porcentual desde línea base (%)')
ax[0].set_ylabel('Frecuencia')

# Boxplot
sns.boxplot(x=df['workplaces'], color='lightblue', ax=ax[1])
ax[1].set_title('Detección de Valores Extremos (Outliers) en Movilidad')
ax[1].set_xlabel('Cambio porcentual desde línea base (%)')

plt.tight_layout()
plt.show()
```



Análisis de Valores Atípicos (Outliers) y Preservación de la Muestra:

El diagnóstico visual mediante diagramas de caja (Boxplots) y distribuciones de densidad revela la presencia de valores extremos en variables como workplaces, daily_cases y los indicadores de movilidad alternativa. Metodológicamente, se ha tomado la decisión estricta de no eliminar ni truncar estas observaciones.

Esto, ya que en el contexto de la crisis sanitaria global (COVID-19), estos outliers no representan errores de medición ni anomalías de digitación, sino que capturan empíricamente los shocks estructurales extremos inherentes a la pandemia (olas de contagio masivo y confinamientos totales). Su eliminación incurriría en un sesgo de selección y censuraría la varianza exacta que el modelo econométrico busca explicar.

1.0.1 Análisis específico por variable

Variable	Valores y Justificación Lógica
<code>iso_code</code>	Valores: 54 categorías únicas (Ej: DE-NW). Justificación: Válido. Corresponde a los identificadores válidos.
<code>retail_and_recreation...</code>	Valores: Mín: -95.60 Máx: 74.60 Media: -16.44. Justificación: Válido. El promedio negativo refleja directamente las restricciones al comercio no esencial durante la pandemia.
<code>grocery_and_pharmacy...</code>	Valores: Mín: -66.20 Máx: 83.60 Media: 5.12. Justificación: Válido. Al ser recintos esenciales para el abastecimiento de comida y medicinas, es lógico que su promedio general sea positivo.
<code>parks_percent...</code>	Valores: Mín: -95.80 Máx: 631.00 Media: 49.68. Justificación: Válido. Los peaks positivos masivos (631%) reflejan el aumento del uso de áreas verdes públicas en las ventanas de desconfinamiento.

Variable	Valores y Justificación Lógica
<code>transit_stations...</code>	Valores: Mín: -88.60 Máx: 123.20 Media: -17.60. Justificación: Válido. Media negativa congruente con la drástica caída en el uso del transporte público por cuarentenas y miedo al contagio.
<code>workplaces_raw</code>	Valores: Mín: -87.00 Máx: 6.60 Media: -25.00. Justificación: Válido. Movilidad laboral sin procesar; la caída de hasta -87% evidencia el shock del teletrabajo y cierres forzosos.
<code>residential_percent...</code>	Valores: Mín: -3.80 Máx: 42.60 Media: 7.07. Justificación: Válido. Tiene sentido que al reducirse la movilidad al trabajo o al comercio, el tiempo transcurrido en el hogar sube (promedio positivo).
<code>trend</code>	Valores: Mín: -80.07 Máx: 12.36 Media: -19.00. Justificación: Válido. Tendencia de movilidad coherente con el resto de las caídas.
<code>workplaces</code>	Valores: Mín: -76.79 Máx: 19.06 Media: -18.67. Justificación: Válido. . Sus promedios negativos capturan correctamente el efecto temporal de los confinamientos laborales.
<code>Valor_Stringency_Index</code>	Valores: Mín: 0.00 Máx: 93.52 Media: 27.49. Justificación: Válido. El índice respeta perfectamente la escala un índice de 0 a 100, sin errores por fuera de ese rango.
<code>Valor_GovernmentResponseIndex</code>	Valores: Mín: 0.00 Máx: 81.31 Media: 30.64. Justificación: Válido. Índice consolidado en márgenes lógicos de 0 a 100.
<code>Valor_EconomicSupportIndex</code>	Valores: Mín: 0.00 Máx: 100.00 Media: 30.96. Justificación: Válido. Se mantiene entre el 0 y 100.
<code>Valor_Containment_Health_index</code>	Valores: Mín: 0.00 Máx: 85.42 Media: 30.59. Justificación: Válido. Captura políticas de testeo y aislamiento dentro de su límite máximo teórico de 100.
<code>workplace_closing</code>	Valores: Mín: 0.00 Máx: 3.00 Media: 0.85. Justificación: Válido. Corresponde exactamente a los 4 niveles categóricos (0, 1, 2, 3) de severidad del cierre de lugares de trabajo definidos en su metodología.
<code>daily_cases</code>	Valores: Mín: 0.00 Máx: 22,203.31 Media: 2,355.73. Justificación: Válido. Es fundamental notar que el mínimo es 0, descartando “contagios negativos”. El máximo refleja el el peak alcanzado.

Variable	Valores y Justificación Lógica
week	Valores: Mín: 1.00 Máx: 140.00 Media: 70.50. Justificación: Válido. Numeración secuencial de las semanas.
year	Valores: Mín: 2020.00 Máx: 2022.00 Media: 2020.97. Justificación: Válido. Son los años más críticos de la crisis del COVID-19 en Europa y el mundo.
NAME	Valores: 86 categorías únicas (Ej: Berlín). Justificación: Válido. Nombres legibles de las ciudades que componen el panel.
Population	Valores: Mín: 201.05 Máx: 10,274.88 Media: 745.47. Justificación: Válido. Población registrada lógicamente sin valores nulos ni negativos, coherente con la demografía de esas 86 regiones.
agriculture	Valores: Mín: 0.00 Máx: 0.07 Media: 0.01. Justificación: Válido. Refleja porcentajes del PIB regional (entre 0% y 7%), que son proporciones habituales para la economía europea.
industry	Valores: Mín: 0.04 Máx: 0.23 Media: 0.12. Justificación: Válido. Peso industrial dentro de valores lógicos.
construction	Valores: Mín: 0.02 Máx: 0.15 Media: 0.05. Justificación: Válido. Proporciones sectoriales que no exceden el límite de 1 (100%).
age_dependency	Valores: Mín: 0.52 Máx: 0.84 Media: 0.65. Justificación: Válido. Ratios demográficos europeos estándar, demostrando que no hay errores de digitación en los censos utilizados.
old_age_dependency	Valores: Mín: 0.26 Máx: 0.49 Media: 0.32. Justificación: Válido. Captura de forma fidedigna la alta tasa de envejecimiento del continente.
young_age_dependency	Valores: Mín: 0.24 Máx: 0.51 Media: 0.33. Justificación: Válido. Cohorte juvenil en proporciones normales.
sex_ratio	Valores: Mín: 91.45 Máx: 115.75 Media: 106.71. Justificación: Válido. Expresa la cantidad de hombres por cada 100 mujeres.
unemp	Valores: Mín: 3.00 Máx: 36.00 Media: 12.22. Justificación: Válido. Tasa de desempleo expresada en porcentaje (0 a 100).

Variable	Valores y Justificación Lógica
f_unemp	Valores: Mín: 0.04 Máx: 0.33 Media: 0.11. Justificación: Válido. Expresado en proporción decimal, coincidente con la variable anterior.
m_unemp	Valores: Mín: 0.02 Máx: 0.43 Media: 0.12. Justificación: Válido. Al igual que el desempleo femenino, está libre de valores negativos o mayores a 1.
foreigners	Valores: Mín: 2.00 Máx: 15.00 Media: 8.15. Justificación: Válido. Porcentaje de población migrante dentro de márgenes lógicos .
country	Valores: 5 categorías únicas (DE, ES, FR, IT, SE). Justificación: Válido. Confirma el espacio muestral de los cinco países definidos para el panel temporal.

Se generan dummies de la variable country y year

```
[12]: # 1. Transformar 'country' en Variables Dummy (Matemáticas 0 y 1)
# drop_first=True elimina el primer país (Alemania 'DE') para evitar
# ↳multicolinealidad perfecta
df = pd.get_dummies(df, columns=['country'], drop_first=True, dtype=int)

# Transformar 'year' en Variables Dummy
df['year'] = df['year'].astype(int).astype(str)
df = pd.get_dummies(df, columns=['year'], drop_first=True, dtype=int)
```

Pregunta 2

Ejecute un modelo Pooled OLS para estimar la relacion entre las restricciones gubernamentales de movilidad y la variacion en movilidad laboral. Seleccione las variables independientes a incluir en el modelo final e interprete su significado.

Correlación con la variable workplaces

```
[13]: # CORRELACIÓN CON LA VARIABLE DEPENDIENTE (WORKPLACES)

import matplotlib.pyplot as plt
import seaborn as sns

# Filtramos las variables numéricas (incluyendo floats e ints por seguridad)
columnas_numericas = df.select_dtypes(include=['float64', 'int64']).columns.
↳tolist()

# Calculamos la correlación de Pearson y aislamos nuestra Y ('workplaces')
correlaciones = df[columnas_numericas].corr()['workplaces'].drop('workplaces',
↳errors='ignore')
```

```

# Ordenamos de la correlación más negativa a la más positiva
corr_ordenada = correlaciones.sort_values()

# Configuración visual del gráfico (Estilo minimalista)
sns.set_theme(style="whitegrid")
plt.figure(figsize=(12, 9))

# Creamos el gráfico de barras.
# Usamos 'hue' y la paleta 'vlag' para mapear colores fríos (negativos) y
↳ cálidos (positivos)
ax = sns.barplot(
    x=corr_ordenada.values,
    y=corr_ordenada.index,
    hue=corr_ordenada.values,
    palette="vlag",
    legend=False,
    edgecolor=".2",
    linewidth=0.8
)

# Anotaciones de los valores numéricos exactos
for i, val in enumerate(corr_ordenada.values):
    # Desplazamiento dinámico para que el texto no pise la barra
    offset = 0.03 if val >= 0 else -0.03
    alineacion = 'left' if val >= 0 else 'right'

    # Formateamos el texto agregando un '+' explícito a los números positivos
    texto_valor = f"{val:+.3f}"

    ax.text(
        val + offset, i,
        texto_valor,
        va='center',
        ha=alineacion,
        fontsize=10,
        fontweight='bold',
        color='#333333'
    )

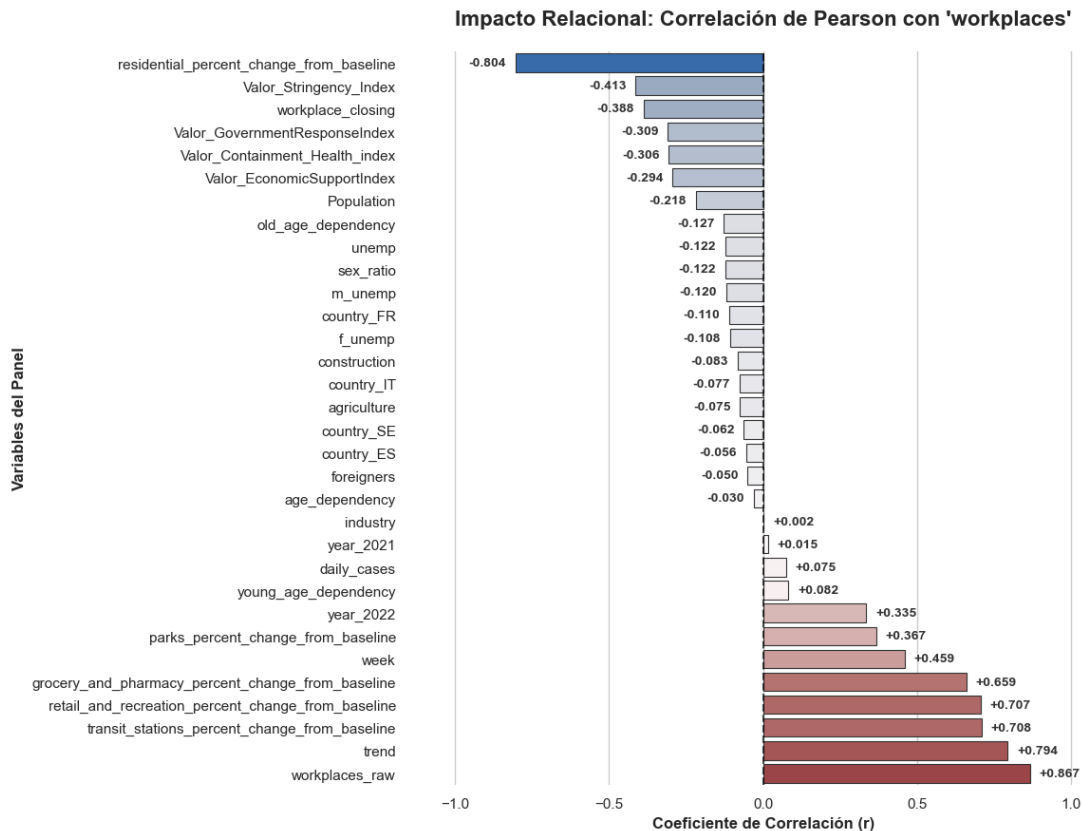
# Estética final de ejes y títulos (CORREGIDO A 'bold')
plt.axvline(0, color='black', linestyle='--', linewidth=1.5, alpha=0.7)
plt.title("Impacto Relacional: Correlación de Pearson con 'workplaces'",
          fontsize=16, fontweight='bold', pad=20)
plt.xlabel('Coeficiente de Correlación (r)', fontsize=12, fontweight='bold')
plt.ylabel('Variables del Panel', fontsize=12, fontweight='bold')

```

```
# Ajustamos los límites de X para asegurar que los textos no se corten
plt.xlim(-1.15, 1.15)

# Eliminamos los bordes superior y derecho de la caja para un look más limpio
sns.despine(left=True, bottom=True)

plt.tight_layout()
plt.show()
```



Matriz de correlación

```
[14]: # Se seleccionan exclusivamente las variables numéricas
# (Para que .corr() no arroje error al intentar leer columnas de texto)
df_numerico = df.select_dtypes(include=[np.number])

# Calculamos la matriz de correlación completa
corr = df_numerico.corr()

# Configuración visual
sns.set(style="white")
f, ax = plt.subplots(figsize=(16, 16)) # Lienzo más grande para que quepan todas
```

```

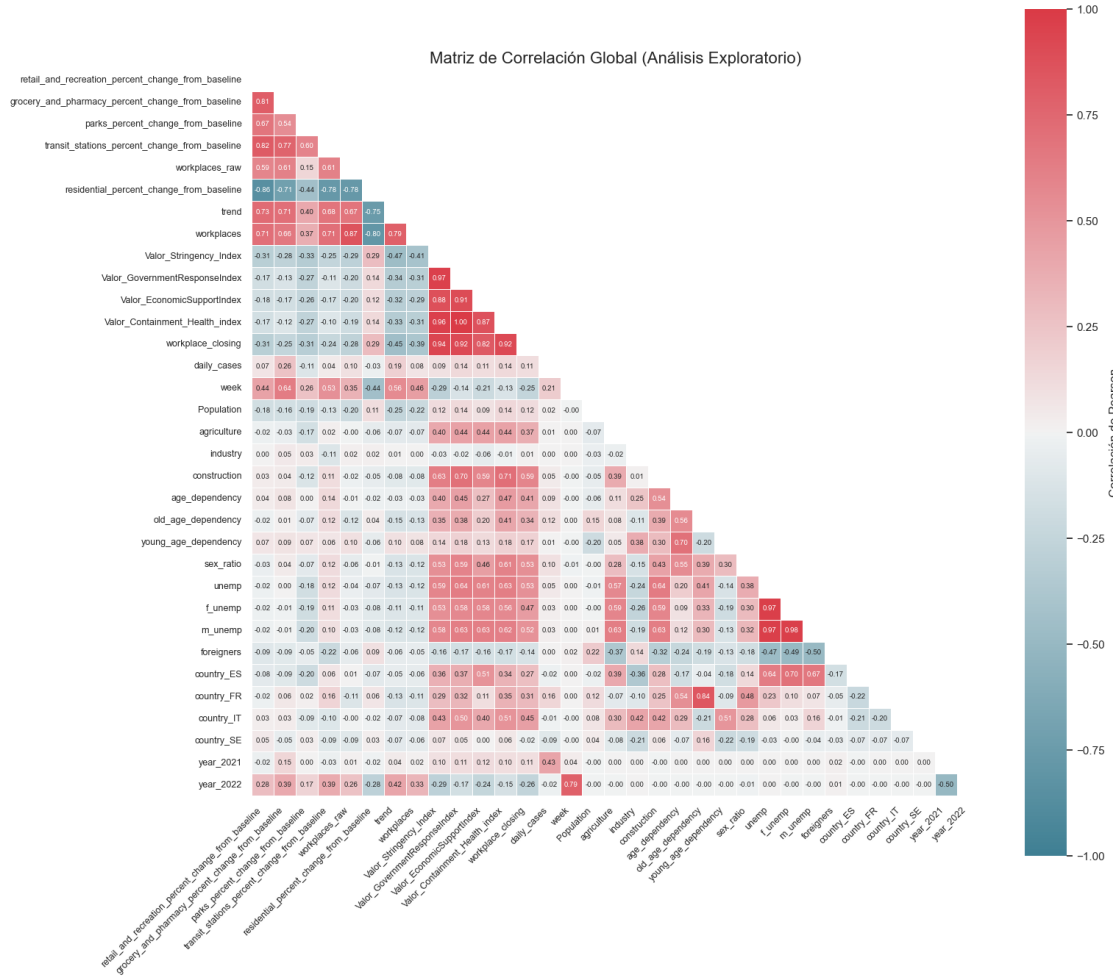
cmap = sns.diverging_palette(220, 10, as_cmap=True)

# Ocultar el triángulo superior para evitar duplicados
mask = np.triu(np.ones_like(corr, dtype=bool))

# Dibujar el mapa de calor
sns.heatmap(
    corr,
    mask=mask,
    cmap=cmap,
    vmax=1.0,
    vmin=-1.0,
    center=0,
    square=True,
    linewidths=0.5,
    cbar_kws={"shrink": 0.8, 'label': 'Correlación de Pearson'},
    annot=True,          # Muestra los valores numéricos
    fmt=".2f",           # Dos decimales
    annot_kws={"size": 7}
)

# Ajustes de legibilidad
plt.xticks(rotation=45, ha='right', fontsize=9)
plt.yticks(fontsize=9)
plt.title('Matriz de Correlación Global (Análisis Exploratorio)', fontsize=16)
plt.tight_layout()
plt.show()

```



Se observa que los índices Stringency, GovernmentResponse, Containment, EconomicsSupport y workplace_closing estaban correlacionados casi al 1, por lo tanto, se debería elegir solo una para que represente la política del gobierno, se escoge Stringency.

f_unamp, m_unamp y unamp también están muy correlacionadas acercándose a 1, por lo que también se elige una de las tres donde la candidata principal en este caso sería unamp que engloba todo el desempleo.

workplaces y workplaces_raw también presentan una alto valor (0.87) ya que se basan lo mismo, por lo tanto se elimina workplaces_raw y se conserva solo workplaces

Las variables que tienen que ver con la movilidad como lo son residential_percent_change_baseline, transit_stations_percent_change_baseline, grocery_and_pharmacy_percent_change_from_baseline, parks_percent_change_from_baseline, retail_and_recreation_percent_change_from_baseline también serán excluidas ya que fueron consecuencia de la cuarentena y serían redundantes, además presentan en su mayoría una alta correlación entre ellas

Para controlar los shocks comunes asociados a la evolución temporal de la pandemia sin saturar

los grados de libertad del modelo base, se prescindió de dummies semanales week. En su lugar, se incorporó el indicador de tendencia continua “trend” y variables dummy anuales con “year”.

A continuacion se ejecuta el VIF para las variables que siguen siendo candidatas

```
[ ]: #TEST DE MULTICOLINEALIDAD GLOBAL (VIF)

from statsmodels.stats.outliers_influence import variance_inflation_factor
import pandas as pd
import statsmodels.api as sm

# 1. Armamos un set de datos amplio con TODAS las X potenciales.

variables_candidatas = [
    'Valor_Stringency_Index',
    #'workplace_closing',
    'daily_cases',
    'Population',
    'agriculture',
    'industry',
    'construction',
    'old_age_dependency',
    'young_age_dependency',
    'age_dependency',
    'sex_ratio',
    'country_ES', 'country_FR', 'country_IT', 'country_SE', 'trend', 'year_2021',
    ↪ 'year_2022',
    'unemp'
]

# Creamos el DataFrame solo con las seleccionadas y eliminamos valores nulos
↪ (vital para el VIF)
X_vif_completo = df[variables_candidatas].dropna().copy()

# Agregamos la constante
X_vif_completo = sm.add_constant(X_vif_completo)

# Calculamos el VIF iterando sobre todas las columnas
vif_data = pd.DataFrame({
    "Variable": X_vif_completo.columns,
    "VIF": [variance_inflation_factor(X_vif_completo.values, i) for i in
    ↪ range(X_vif_completo.shape[1])]
})

# Mostramos ordenado de mayor a menor (omitiendo la constante)
tabla_vif = vif_data[vif_data['Variable'] != 'const'].sort_values("VIF",
    ↪ ascending=False)
display(tabla_vif.round(2))
```

	Variable	VIF
9	age_dependency	1396.27
8	young_age_dependency	965.73
7	old_age_dependency	674.49
12	country_FR	11.47
13	country_IT	7.19
11	country_ES	6.44
18	unemp	4.78
1	workplace_closing	4.18
6	construction	3.56
10	sex_ratio	2.84
5	industry	2.09
17	year_2022	2.03
15	trend	1.98
14	country_SE	1.92
16	year_2021	1.90
4	agriculture	1.84
2	daily_cases	1.43
3	Population	1.40

Se pueden observar que hay variables con muy alto VIF por lo que se deben ir eliminando para evitar las redundancias. Con estos resultados iniciales se puede observar problemas de multicolinealidad, especialmente entre las variables demográficas. Dado que `young_age_dependency` y `old_age_dependency` son subcomponentes directos de la tasa de dependencia demográfica global, su inclusión simultánea genera redundancia.

Es por esto que se eliminará `young_age_dependency` y `old_age_dependency` y se dejará `age_dependency`.

La variable `trend` será excluida de la especificación final de los modelos debido a que representa una tendencia temporal agregada del periodo analizado, capturando la evolución general del tiempo durante la pandemia. Si bien su inclusión incrementa considerablemente el ajuste estadístico del modelo, también absorbe gran parte de la variación temporal asociada a las restricciones gubernamentales y a la dinámica propia del COVID-19, dificultando la interpretación de las variables de interés. Además, `trend` no corresponde a una variable económica claramente definida, no representa una política pública específica, no constituye una característica estructural observable de las regiones y carece de una interpretación causal natural en relación con la movilidad laboral.

Se observará el efecto de esto y luego se continuará si es necesario.

[16]: *#TEST DE MULTICOLINEALIDAD GLOBAL (VIF)*

```
variables_candidatas = [
    'Valor_Stringency_Index',
    #'workplace_closing',
    'daily_cases',
    'Population',
    'agriculture',
```



```

    'industry',
    'construction',
    'age_dependency',
    'sex_ratio',
    'country_ES',
    'country_FR',
    'country_IT',
    'country_SE',
    #'trend',
    #'year_2021',
    #'year_2022',
    'unemp',
]

# Creamos el DataFrame solo con las seleccionadas y eliminamos valores nulos
↳ (vital para el VIF)
X_vif_completo = df[variables_candidatas].dropna().copy()

# 2. Agregamos la constante (Paso obligatorio para el cálculo matricial)
X_vif_completo = sm.add_constant(X_vif_completo)

# 3. Calculamos el VIF iterando sobre todas las columnas
vif_data = pd.DataFrame({
    "Variable": X_vif_completo.columns,
    "VIF": [variance_inflation_factor(X_vif_completo.values, i) for i in
↳ range(X_vif_completo.shape[1])]
})

# 4. Mostramos ordenado de mayor a menor (omitiendo la constante)
tabla_vif = vif_data[vif_data['Variable'] != 'const'].sort_values("VIF",
↳ ascending=False)
display(tabla_vif.round(2))

```

	Variable	VIF
9	country_ES	5.47
11	country_IT	5.39
10	country_FR	5.22
13	unemp	4.28
1	Valor_Stringency_Index	3.56
6	construction	3.39
7	age_dependency	2.78
8	sex_ratio	2.75
5	industry	1.84
4	agriculture	1.79
12	country_SE	1.38
3	Population	1.18
2	daily_cases	1.04

Estas eliminaciones tuvieron buenos efectos, logrando reducir el VIF de `age_dependency` a 2.78 y disminuyendo drásticamente la multicolinealidad general. Si bien algunas variables de control como las `country` muestran un VIF levemente superior a 5, este valor sigue encontrándose en un rango aceptable y no compromete la variable principal de estudio.

Para formar el panel de datos primero se ajusta la columna de fechas a un formato de tiempo. Luego, se establece la ciudad (`CODE`) y la fecha (`date`) como el índice principal. Se utilizó `CODE` ya que es un código alfanumérico, sin espacios ni caracteres especiales como los puede tener `name` o `iso-code`.

Para responder a la pregunta, el objetivo es aislar el impacto de las políticas gubernamentales sobre la movilidad laboral, controlando por todos los demás factores que también podrían haber dejado a la gente en sus hogares. Tras el análisis presentado anteriormente, el modelo final se compone de las siguientes variables:

Variables	Justificación
workplaces (Variable dependiente)	Representa la variación porcentual de la movilidad hacia los lugares de trabajo respecto a la línea base
Valor_Stringency_Index	Índice continuo 0-100 que expresa directamente que expresa directamente la severidad de las restricciones gubernamentales de movilidad
Shock Sanitario (daily_cases)	Las personas pueden dejar de ir al trabajo voluntariamente por el pánico a los contagios, que es de manera independientemente de lo que puede definir el gobierno
Matriz Productiva (agriculture, industry, construction)	Regiones con alta concentración agrícola o de construcción no pueden aplicar “teletrabajo” lo que influya en la variación en los lugares de trabajo
Estructura Demográfica (Population, age_dependency, sex_ratio)	Las tasas de dependencia dictan la proporción real de la población que pertenece a la fuerza laboral activa
Desempleo (unemp)	La tasa de desempleo afecta estructuralmente la proporción de personas que se movilizan al trabajo
Países (country_ES, country_IT, country_FR, country_SE)	Controlan las diferencias culturales, legales y económicas propias de cada nación, usando a Alemania como punto de comparación.

[17]: `# Definición de la estructura temporal-espacial del Panel (Set Index)`

```
dfpanel = df.set_index(['CODE', 'date'])
```

[18]: `#MODELO POOLED OLS (Mínimos Cuadrados Agrupados)`

```

# Definimos nuestras variables finales seleccionadas
variables_finales = [
    'Valor_Stringency_Index',

    'daily_cases',      # Control: Shock Sanitario
    'Population',       # Control: Demografía
    'agriculture',      # Control: Matriz Productiva
    'industry',
    'construction',
    'age_dependency',   # Control: Dependencia demográfica
    'sex_ratio',        # Control: Composición por sexo
    'country_ES',       # Control Geográfico: España
    'country_FR',       # Control Geográfico: Francia
    'country_IT',       # Control Geográfico: Italia
    'country_SE',       # Control Geográfico: Suecia
    #'trend',           # Control Temporal: Tendencia continua
    #'year_2021',       # Control Temporal: Año 2021
    #'year_2022'        # Control Temporal: Año 2022
    'unemp'
]

# Limpieza de datos nulos estrictamente en las columnas del modelo
datos_modelo = dfpanel[['workplaces'] + variables_finales].dropna()

y = datos_modelo['workplaces']
X = datos_modelo[variables_finales]

# Inclusión del Intercepto matemático (\beta_0)
X = sm.add_constant(X)

# Estimación del Modelo Pooled OLS
# Utilizamos cov_type='robust' para blindar el modelo contra la
↳ heterocedasticidad
modelo_pooled = lmp.PooledOLS(y, X)
resultados_pooled = modelo_pooled.fit(cov_type='robust')

# Despliegue de Resultados
print(resultados_pooled)
#print(resultados_pooled.summary)

```

PooledOLS Estimation Summary

```

=====
Dep. Variable:          workplaces    R-squared:                0.3191
Estimator:              PooledOLS    R-squared (Between):      0.6349
No. Observations:      12040         R-squared (Within):       0.2672
Date:                   Mon, May 25 2026   R-squared (Overall):      0.3191
Time:                   20:49:34          Log-likelihood            -4.449e+04

```

Cov. Estimator:	Robust	F-statistic:	433.49
Entities:	86	P-value	0.0000
Avg Obs:	140.00	Distribution:	F(13,12026)
Min Obs:	140.00		
Max Obs:	140.00	F-statistic (robust):	285.56
		P-value	0.0000
Time periods:	140	Distribution:	F(13,12026)
Avg Obs:	86.000		
Min Obs:	86.000		
Max Obs:	86.000		

Parameter Estimates

	Parameter	Std. Err.	T-stat	P-value	Lower CI
Upper CI					
const	-2.4075	3.4818	-0.6915	0.4893	-9.2324
4.4173					
Valor_Stringency_Index	-0.3502	0.0072	-48.949	0.0000	-0.3643
-0.3362					
daily_cases	0.0005	2.503e-05	18.750	0.0000	0.0004
0.0005					
Population	-0.0017	7.638e-05	-21.745	0.0000	-0.0018
-0.0015					
agriculture	-24.178	12.675	-1.9075	0.0565	-49.024
0.6674					
industry	-23.252	2.6747	-8.6932	0.0000	-28.494
-18.009					
construction	29.371	9.5333	3.0809	0.0021	10.684
48.058					
age_dependency	18.532	2.1348	8.6810	0.0000	14.347
22.716					
sex_ratio	-0.2227	0.0336	-6.6377	0.0000	-0.2885
-0.1569					
country_ES	13.915	0.5707	24.381	0.0000	12.796
15.033					
country_FR	9.9453	0.5640	17.634	0.0000	8.8399
11.051					
country_IT	16.115	0.5816	27.709	0.0000	14.975
17.255					
country_SE	5.7235	0.7875	7.2680	0.0000	4.1799
7.2671					
unemp	-0.0534	0.0233	-2.2967	0.0217	-0.0990
-0.0078					

=====

Conclusiones del Modelo Pooled OLS (Nueva Especificación):

Poder Explicativo del Modelo (R^2): El modelo logra explicar un 31.91% de la varianza total de la variación en la movilidad laboral (R^2 Overall). Desglosando este poder predictivo, el modelo es particularmente bueno para explicar las diferencias estructurales entre las distintas ciudades con un 63.49% (R^2 Between), pero explica en menor medida las variaciones que ocurren a través del tiempo dentro de cada ciudad, con un 26.72% (R^2 Within).

Efecto de las variables en la variación de la movilidad laboral:

Restricciones Gubernamentales (Valor_Stringency_Index): Su efecto es negativo y altamente significativo (p-value < 0.001). Si el índice de rigurosidad aumenta en 1 punto (medidas más estrictas), se asocia con una fuerte disminución promedio de 0.3502 puntos porcentuales en la movilidad laboral.

Tasa de Desempleo (unemp): Resultó ser estadísticamente significativa (p-value = 0.0217). Un aumento de un 1% en la tasa de desempleo local reduce la variación de movilidad en 0.0534 puntos porcentuales, confirmando que a mayor desempleo, una menor proporción de la población se desplaza hacia lugares de trabajo.

Controles Geográficos (Country): Se utilizó a Alemania como país base de comparación. Las variables dummy para España, Francia, Italia y Suecia son positivas y significativas. Esto indica que, controlando por todas las demás variables, Italia por ejemplo tiene en promedio una variación de movilidad laboral 16.115 puntos porcentuales mayor que Alemania.

Matriz Productiva (agriculture, industry, construction): Tienen un impacto altamente significativo. Ciudades con un peso mayoritario en la industria experimentan una disminución relativa en su movilidad (-23.252 puntos), mientras que ciudades orientadas a la construcción ven un aumento en su movilidad (+29.371 puntos), posiblemente porque el rubro de la construcción es incompatible con el teletrabajo.

Estructura Demográfica: El aumento poblacional (Population) y la proporción de hombres (sex_ratio) empujan levemente la movilidad hacia la baja. Por otro lado, la tasa de dependencia (age_dependency) tiene un fuerte impacto positivo (+18.532) sobre la movilidad observada.

Shock Sanitario (daily_cases): Muestra un coeficiente significativo pero ínfimo (0.0005). Este valor, contraintuitivamente positivo, advierte la posible presencia de un sesgo por variable omitida (por ejemplo, al no incluir tendencias temporales o estacionales, los peaks de contagios pueden estar coincidiendo numéricamente con meses de mayor movilidad basal).

Sobre la Constante: La constante del modelo no es estadísticamente significativa (p-value=0.4893). Esto indica que no existe evidencia suficiente para afirmar que el intercepto difiere de cero. Dado que representa el valor esperado de movilidad para una ciudad en Alemania (país base) cuando todas las métricas son cero, su falta de significancia es matemáticamente correcta y no afecta la interpretación de nuestras variables de interés.

La constante del modelo no resulta estadísticamente significativa (p-value=0.6043) lo que esta indicando que no existe evidencia suficiente para afirmar que el intercepto difiere de cero una vez controladas las demás variables del modelo. Dado que la constante representa el valor esperado de la variación de movilidad laboral cuando los regresores toman valor cero, su falta de significancia no

afecta la interpretación de las variables de interés. Metodológicamente, se mantiene el intercepto para evitar forzar la regresión por el origen y preservar la correcta estimación de los coeficientes.

Pregunta 3

Se continua con las variables seleccionadas previamente para la posterior comparación entre modelos. Se genera el modelo de efectos fijos a continuación.

```
[19]: modeloFE = lmp.PanelOLS(y, X, entity_effects=True, drop_absorbed=True) #
      fe = modeloFE.fit(cov_type="robust")
      print(fe)
```

```

                                PanelOLS Estimation Summary
=====
Dep. Variable:                workplaces    R-squared:                0.2700
Estimator:                    PanelOLS      R-squared (Between):      -763.55
No. Observations:              12040        R-squared (Within):       0.2700
Date:                          Mon, May 25 2026  R-squared (Overall):     -107.52
Time:                          20:49:34        Log-likelihood            -4.399e+04
Cov. Estimator:                Robust

                                F-statistic:                736.37
Entities:                      86              P-value                  0.0000
Avg Obs:                       140.00          Distribution:             F(6,11948)
Min Obs:                       140.00
Max Obs:                       140.00          F-statistic (robust):     521.47
                                P-value                  0.0000
Time periods:                  140          Distribution:             F(6,11948)
Avg Obs:                       86.000
Min Obs:                       86.000
Max Obs:                       86.000

```

```

                                Parameter Estimates
=====
=====
                                Parameter  Std. Err.    T-stat    P-value    Lower CI
Upper CI
-----
const                -152.14      51.963    -2.9277    0.0034    -253.99
-50.279
Valor_Stringency_Index -0.3509     0.0074    -47.165    0.0000    -0.3655
-0.3363
daily_cases          0.0005     2.459e-05    18.522    0.0000     0.0004
0.0005
Population           -0.0964     0.0218    -4.4278    0.0000    -0.1390
-0.0537
industry             -257.83     28.643    -9.0016    0.0000    -313.97
-201.69
sex_ratio             2.2099     0.4771     4.6316    0.0000     1.2746

```

3.1452					
unemp	0.6381	0.1702	3.7500	0.0002	0.3046
0.9717					

=====

=====

F-test for Poolability: 30.285
P-value: 0.0000
Distribution: F(85,11948)

Included effects: Entity

Conclusiones del Modelo de Efectos Fijos (FE):

Para el modelo de efectos fijos se tomó la variación de movilidad laboral (workplaces) como variable dependiente. La librería identificó y absorbió automáticamente las variables que eran constantes en el tiempo para cada ciudad (como las dummies de los países, agriculture y construction), dejando únicamente aquellas que presentaban fluctuaciones temporales.

Las conclusiones principales son:

Poder Explicativo y Validez del Modelo: El modelo logra explicar un 27.00% de la varianza total de la movilidad a lo largo del tiempo dentro de cada ciudad (R^2 Within). Los valores negativos en el R^2 Overall y Between son comunes matemáticamente en estimaciones de Efectos Fijos con datos muy heterogéneos, ya que el modelo se optimiza exclusivamente para capturar la variabilidad temporal.

Efecto de las variables en la movilidad temporal

Dado que todas las variables retenidas son estadísticamente significativas ($p - value < 0.001$), se concluye lo siguiente:

Restricciones Gubernamentales (Valor_Stringency_Index): Un aumento temporal de 1 punto en el índice de rigurosidad reduce directamente la movilidad laboral en 0.3509 puntos porcentuales.

Tasa de Desempleo (unemp): Al analizar la variación dentro de la ciudad, un aumento temporal en la tasa de desempleo se asocia a un aumento temporal en la movilidad de 0.6381 puntos (pasa de negativo en OLS a positivo en FE), en ese sentido los periodos de mayor desempleo en una ciudad pueden forzar a la población a salir de sus casas para buscar trabajo activamente de forma presencial o incurrir en informalidad.

Variables demograficas (Population, sex_ratio): Aunque pueden pensarse como estáticas, en esta base de datos presentaron variación temporal. Aumentos de población en el tiempo empujan levemente la movilidad a la baja, mientras que los cambios en la proporción por sexo la empujan al alza.

Shock Sanitario (daily_cases): Se mantiene altamente significativo y con coeficiente positivo, pero con un impacto que se hace notar al tener miles de contagios lo cual como se señaló antes es contraintuitivo.

Industry: La concentración industrial cambiará muy poco en espacios de tiempo pequeños, como su variación temporal es tan pequeña, ese -257.83 como resultado puede ser esperado matemáticamente

en Efectos Fijos al tratarse de una variable estructural de lento cambio, por lo que su interpretación económica directa debe tomarse con precaución.

Pregunta 4

Ejecute un modelo de efectos aleatorios para estimar la misma relacion anterior. Seleccione las variables independientes a incluir en el modelo final e interprete su significado.

```
[20]: modeloRE=lmpr.RandomEffects(y,X)
re=modeloRE.fit(cov_type="robust")
print(re)
```

```

RandomEffects Estimation Summary
=====
Dep. Variable:          workplaces    R-squared:                0.2716
Estimator:              RandomEffects  R-squared (Between):      0.6284
No. Observations:      12040          R-squared (Within):       0.2673
Date:                  Mon, May 25 2026  R-squared (Overall):     0.3183
Time:                  20:49:35          Log-likelihood            -4.405e+04
Cov. Estimator:        Robust

                               F-statistic:          345.00
Entities:              86                P-value              0.0000
Avg Obs:               140.00            Distribution:         F(13,12026)
Min Obs:               140.00
Max Obs:               140.00            F-statistic (robust):   210.74
                               P-value              0.0000
Time periods:          140              Distribution:         F(13,12026)
Avg Obs:               86.000
Min Obs:               86.000
Max Obs:               86.000

```

```

Parameter Estimates
=====
=====

```

	Parameter	Std. Err.	T-stat	P-value	Lower CI
Upper CI					

const	-14.753	11.871	-1.2428	0.2140	-38.022
8.5153					
Valor_Stringency_Index	-0.3507	0.0070	-49.895	0.0000	-0.3644
-0.3369					
daily_cases	0.0005	2.422e-05	19.385	0.0000	0.0004
0.0005					
Population	-0.0016	0.0003	-6.0113	0.0000	-0.0022
-0.0011					
agriculture	-27.343	45.946	-0.5951	0.5518	-117.40
62.718					
industry	-19.571	9.4511	-2.0708	0.0384	-38.097

-1.0454					
construction	40.602	31.546	1.2871	0.1981	-21.234
102.44					
age_dependency	15.589	7.3996	2.1068	0.0352	1.0848
30.094					
sex_ratio	-0.0951	0.1148	-0.8281	0.4077	-0.3201
0.1300					
country_ES	13.281	1.6980	7.8218	0.0000	9.9529
16.610					
country_FR	9.1286	1.6671	5.4756	0.0000	5.8608
12.396					
country_IT	15.179	1.5966	9.5076	0.0000	12.050
18.309					
country_SE	6.0112	2.3937	2.5113	0.0120	1.3192
10.703					
unemp	-0.0504	0.0813	-0.6208	0.5347	-0.2097
0.1088					
=====					
=====					

Conclusiones del Modelo de Efectos Aleatorios (RE):

Para el modelo de efectos aleatorios se tomó la variación en la movilidad laboral (workplaces) como variable dependiente, utilizando el mismo conjunto de variables independientes que en los modelos anteriores.

Las conclusiones principales son:

Poder Explicativo del Modelo (R^2): El modelo logra explicar un 31.83% de la varianza total de la movilidad laboral en el contexto general (R^2 Overall). De manera similar al Pooled OLS, logra explicar de muy buena forma (en un 62.84%) la varianza estructural entre las distintas ciudades (R^2 Between), y explica de manera moderada (en un 26.73%) los cambios de variabilidad a lo largo del tiempo (R^2 Within).

Efecto de las variables en la variación de la movilidad laboral:

Restricciones Gubernamentales (Valor_Stringency_Index): Mantiene un efecto negativo, si el índice aumenta en 1 punto, se asocia con una disminución promedio de 0.3507 puntos porcentuales en la movilidad laboral.

Matriz Productiva (industry): Tener una alta dependencia en el sector industrial reduce la expectativa de movilidad laboral en 19.571 puntos. (Los sectores de agricultura y construcción perdieron significancia estadística en este modelo).

Estructura Demográfica: Un aumento en la tasa de dependencia demográfica (age_dependency) aumenta la movilidad base en 15.589 puntos. Por el contrario, ciudades con mayor volumen de población (Population) ven disminuida levemente su movilidad en 0.0016 puntos por unidad poblacional.

Controles Geográficos (Países): Utilizando a Alemania como base, todas las demás naciones analizadas incrementan la expectativa de movilidad: España en 13.281 puntos, Francia en 9.128 puntos,

Italia en 15.179 puntos y Suecia en 6.011 puntos. Esto revela diferencias culturales o macroeconómicas persistentes entre países frente a la pandemia.

Shock Sanitario (daily_cases): Se mantiene altamente significativo, pero con un impacto que como se señaló anteriormente se hace notar cuando hay miles de casos diarios.

Variables no significativas: Bajo la estructura de Efectos Aleatorios, la tasa de desempleo (unemp), la proporción de hombres (sex_ratio), y sectores como agriculture y construction demostraron tener un impacto no significativo ($p - \text{values} > 0.05$) en la movilidad laboral.

Pregunta 5

Comente los resultados obtenidos en 2, 3 y 4. ¿Cuáles y por qué existen las diferencias entre los resultados?. En su opinión, ¿Cuál sería el más adecuado para responder la pregunta de investigación y por qué? ¿Qué variables resultaron ser robustas a la especificación?

Se compararan los tres modelos a través de la tabla presentada a continuación, junto con la aplicación del test de Hausman

```
[21]: # Comparar los tres modelos en una sola tabla
print("  Comparación de Modelos Panel  ")
print(lmp.compare({'Pooled OLS': resultados_pooled, 'Fixed Effects': fe,
                  ↪ 'Random Effects': re}))

def hausman(fe, re, verbose=True):
    common = fe.params.index.intersection(re.params.index)

    # Filtrar constante
    common = [c for c in common if c.lower() != 'const']

    # Es mejor restar FE - RE para mantener consistencia
    b_diff = fe.params[common] - re.params[common]

    v_diff = fe.cov.loc[common, common] - re.cov.loc[common, common]

    if b_diff.isnull().any() or v_diff.isnull().values.any():
        raise ValueError("NaNs en b_diff o v_diff")

    cond = np.linalg.cond(v_diff)
    if cond > 1e12:
        if verbose:
            print(f" Matriz casi singular (condición = {cond:.2e}), usando_
            ↪ pseudoinversa")
        v_inv = np.linalg.pinv(v_diff)
    else:
        v_inv = np.linalg.inv(v_diff)

    W = float(b_diff.T @ v_inv @ b_diff)
    df = len(common)
    pval = stats.chi2.sf(W, df)
```

```

    if verbose:
        decision = " Rechazar H0 → usar FE" if pval < 0.05 else " No se rechaza H0 → usar RE"
        print(f"Hausman Test:  $\chi^2({df}) = {W:.2f}$ , p-value = {pval:.4f} → {decision}")

    return W, df, pval

hausman(fe, re)

```

Comparación de Modelos Panel

Model Comparison

	Pooled OLS	Fixed Effects	Random Effects
Dep. Variable	workplaces	workplaces	workplaces
Estimator	PooledOLS	PanelOLS	RandomEffects
No. Observations	12040	12040	12040
Cov. Est.	Robust	Robust	Robust
R-squared	0.3191	0.2700	0.2716
R-Squared (Within)	0.2672	0.2700	0.2673
R-Squared (Between)	0.6349	-763.55	0.6284
R-Squared (Overall)	0.3191	-107.52	0.3183
F-statistic	433.49	736.37	345.00
P-value (F-stat)	0.0000	0.0000	0.0000
const	-2.4075 (-0.6915)	-152.14 (-2.9277)	-14.753 (-1.2428)
Valor_Stringency_Index	-0.3502 (-48.949)	-0.3509 (-47.165)	-0.3507 (-49.895)
daily_cases	0.0005 (18.750)	0.0005 (18.522)	0.0005 (19.385)
Population	-0.0017 (-21.745)	-0.0964 (-4.4278)	-0.0016 (-6.0113)
agriculture	-24.178 (-1.9075)		-27.343 (-0.5951)
industry	-23.252 (-8.6932)	-257.83 (-9.0016)	-19.571 (-2.0708)
construction	29.371 (3.0809)		40.602 (1.2871)
age_dependency	18.532 (8.6810)		15.589 (2.1068)
sex_ratio	-0.2227 (-6.6377)	2.2099 (4.6316)	-0.0951 (-0.8281)
country_ES	13.915 (24.381)		13.281 (7.8218)

country_FR	9.9453		9.1286
	(17.634)		(5.4756)
country_IT	16.115		15.179
	(27.709)		(9.5076)
country_SE	5.7235		6.0112
	(7.2680)		(2.5113)
unemp	-0.0534	0.6381	-0.0504
	(-2.2967)	(3.7500)	(-0.6208)
=====			
Effects		Entity	

T-stats reported in parentheses

Matriz casi singular (condición = 2.01e+14), usando pseudoinversa

Hausman Test: $\chi^2(6) = 133.40$, p-value = 0.0000 → Rechazar H0 → usar FE

[21]: (133.4036951435202, 6, np.float64(2.4661611563841492e-26))

Existen diferencias en los resultados de los tres modelos debido a que cada uno hace supuestos distintos sobre la naturaleza de los datos:

El modelo Pooled OLS es una regresión simple que ignora por completo la estructura de panel, asumiendo que todas las observaciones son independientes y omitiendo el componente temporal e individual.

El modelo de Efectos Fijos (FE) considera la estructura de panel y asume que existen efectos inobservados propios y constantes para cada ciudad (como su cultura, geografía o infraestructura) que sí están correlacionados con las variables explicativas. Por ende, aísla estos efectos para estimar puramente el impacto de la variación temporal.

El modelo de Efectos Aleatorios (RE) supone que esos efectos inobservados de cada ciudad son simplemente aleatorios y que no tienen relación (no están correlacionados) con las variables explicativas.

Conclusiones de la selección de modelo:

Descarte de OLS: El modelo Pooled OLS se descarta porque ignora la estructura longitudinal de los datos y asume ingenuamente que todas las ciudades son idénticas en su comportamiento basal.

Test de Hausman

Para decidir matemáticamente entre Efectos Fijos y Efectos Aleatorios, se aplica el Test de Hausman. El resultado obtenido es contundente: p-value = 0.0000 → Rechazar H0. Al rechazar la hipótesis nula, se concluye que los efectos no observados si están correlacionados con los regresores. Por lo tanto, el modelo de Efectos Aleatorios entrega estimadores inconsistentes y el modelo que se debería utilizar es el de Efectos Fijos.

Modelo FE: El modelo de Efectos Fijos arrojó estimadores altamente significativos (todos con p-value < 0.001) y un R^2 Within robusto del 27.00%. Por lo tanto, reponiendo a la pregunta de investigación, se concluye estadísticamente que las políticas públicas tienen un efecto causal y directo en el comportamiento de la población, controlando por toda la heterogeneidad inobservada de cada ciudad, un aumento de 1 punto en el índice de restricciones gubernamentales

(Valor_Stringency_Index) genera una caída de 0.3509 puntos porcentuales en la movilidad laboral. Este impacto resulta ser el mecanismo para reducir la movilidad, muy por encima del confinamiento voluntario generado por el pánico a los contagios diarios (daily_cases).

Pregunta 6

Ejecute un modelo de efectos aleatorios correlacionados (CRE) para estimar la misma relación anterior. Seleccione las variables independientes a incluir en el modelo final e interprete su significado. Es este modelo adecuado, dada la data disponible, para modelar el componente no observado?

```
[22]: # PREGUNTA 6: MODELO DE EFECTOS ALEATORIOS CORRELACIONADOS (CRE / MUNDLAK)

import statsmodels.api as sm
from linearmodels.panel import RandomEffects
import pandas as pd

# 1. Variables explicativas dinámicas (Time-Varying): Todas las que fluctúan en
    ↪ el tiempo
X_dinamicas = [
    'Valor_Stringency_Index',
    'daily_cases',
    'unemp',
    'Population',
    'sex_ratio'
]

# 2. Variables explicativas de control estructural (Time-Invariant): Las
    ↪ constantes
X_estaticas = [
    'agriculture', 'industry', 'construction', 'age_dependency',
    'country_ES', 'country_FR', 'country_IT', 'country_SE'
]

# 3. Usamos DF_PANEL (vital para que funcione) para calcular las medias por
    ↪ ciudad (level=0)
Xm = dfpanel.groupby(level=0)[X_dinamicas].transform('mean')
Xm.columns = ['mean_' + col for col in Xm.columns]

# 4. Unimos la tabla de medias al dfpanel
dfpanel_cre = pd.concat([dfpanel, Xm], axis=1)

# 5. Definimos el conjunto final de regresores
variables_finales_cre = X_dinamicas + X_estaticas + list(Xm.columns)

# Limpieza de nulos
datos_modelo_cre = dfpanel_cre[['workplaces'] + variables_finales_cre].dropna()

y_cre = datos_modelo_cre['workplaces']
```

```

X_cre = datos_modelo_cre[variables_finales_cre]

# 6. Añadimos constante
X_cre = sm.add_constant(X_cre)

# 7. Estimación del Modelo CRE
cre_model = RandomEffects(y_cre, X_cre, check_rank=False)
cre_res = cre_model.fit(cov_type='robust')

# Despliegue de resultados
print(cre_res)

```

RandomEffects Estimation Summary

```

=====
Dep. Variable:          workplaces    R-squared:                0.2739
Estimator:              RandomEffects  R-squared (Between):      0.6345
No. Observations:      12040          R-squared (Within):       0.2699
Date:                  Mon, May 25 2026  R-squared (Overall):      0.3213
Time:                  20:49:36          Log-likelihood            -4.403e+04
Cov. Estimator:        Robust

                                F-statistic:          251.97
Entities:              86                P-value              0.0000
Avg Obs:               140.00            Distribution:         F(18,12021)
Min Obs:               140.00
Max Obs:               140.00            F-statistic (robust):    156.26
                                P-value              0.0000
Time periods:          140              Distribution:         F(18,12021)
Avg Obs:               86.000
Min Obs:               86.000
Max Obs:               86.000

```

Parameter Estimates

```

=====
=====

```

	Parameter	Std. Err.	T-stat	P-value	Lower	Upper CI
const	-2.1280					
Valor_Stringency_Index	-0.3512	0.0074	-47.218	0.0000	-0.3658	-0.3367
daily_cases	0.0005	2.451e-05	18.587	0.0000	0.0004	0.0005
unemp	0.6407	0.2210	2.8989	0.0038	0.2075	1.0739
Population	-0.0946	0.0216	-4.3904	0.0000	-0.1369	-0.0524
sex_ratio	2.2113	0.4774	4.6316	0.0000		

1.2755	3.1472				
agriculture		-22.359	47.458	-0.4711	0.6376
-115.38	70.666				
industry		-23.857	9.8701	-2.4171	0.0157
-43.204	-4.5103				
construction		37.986	32.636	1.1639	0.2445
-25.986	101.96				
age_dependency		18.021	7.7097	2.3374	0.0194
2.9085	33.133				
country_ES		1.8456			
country_FR		-1.5462			
country_IT		2.2765			
country_SE		-3.7201			
mean_Valor_Stringency_Index		0.2420	6.762e+04	3.579e-06	1.0000
-1.325e+05	1.325e+05				
mean_daily_cases		0.0003			
mean_unemp		-0.7040	0.2364	-2.9777	0.0029
-1.1674	-0.2406				
mean_Population		0.0930	0.0215	4.3192	0.0000
0.0508	0.1352				
mean_sex_ratio		-2.4411	0.4950	-4.9318	0.0000
-3.4114	-1.4709				
=====					
=====					

Conclusiones del Modelo de Efectos Aleatorios Correlacionados (CRE):

Para el modelo CRE se tomó la variación de movilidad laboral (workplaces) como variable dependiente. Como variables independientes se utilizaron las mismas que en los modelos anteriores, con la diferencia metodológica de que se le concatenaron los promedios temporales (mean_) de cada variable que fluctuaba en el tiempo para cada ciudad.

Las conclusiones principales son:

Poder Explicativo del Modelo (R^2): El modelo logra explicar un 32.13% de la varianza total de la movilidad laboral en el contexto general (R^2 Overall). Logra explicar de buena forma la varianza estructural entre distintas ciudades con un 63.45% (R^2 Between), y explica de manera moderada los cambios temporales dentro de cada ciudad con un 26.99% (R^2 Within). Estos valores son muy similares a los obtenidos en los modelos anteriores, demostrando la robustez de la base de datos.

Efectos temporales dentro de cada ciudad (Variables Originales):

Restricciones Gubernamentales (Valor_Stringency_Index): Se mantiene como el factor más robusto de todo el estudio. El modelo aísla su efecto puro, concluyendo que al aumentar la severidad en 1 punto, la movilidad laboral de la ciudad cae temporalmente en 0.3512 puntos porcentuales (p-value = 0.0000). Tasa de desempleo (unemp): Refleja la variación temporal. Un aumento en la tasa de desempleo a lo largo del tiempo dentro de la misma ciudad se asocia a un alza de 0.6407 en la movilidad laboral, posiblemente por el aumento en la búsqueda presencial de empleo. Población y Sexo: Un aumento temporal en la población (Population) muestra una leve disminución temporal en la movilidad (-0.0946), mientras que cambios en la demografía por sexo (sex_ratio) aumentan la movilidad (2.2113). Shock Sanitario (daily_cases): Se mantiene marginalmente positivo e ínfimo

(0.0005). Efectos estructurales constantes (Matriz Productiva y Demografía Estática):

Al igual que en Efectos Aleatorios, las ciudades con alta dependencia en la industria (industry) presentan una movilidad base 23.857 puntos menor que el resto (p-value = 0.0157). Las ciudades con altas tasas de dependencia demográfica (age_dependency) muestran una movilidad base mayor en 18.021 puntos. Los rubros de agriculture y construction resultaron no ser estadísticamente significativos. Efectos de la media de las variables para captar las diferencias entre ciudades (mean_):

Media de desempleo (mean_unemp): Ciudades que en promedio tienen un nivel de desempleo 1 punto mayor que otras, experimentan una movilidad estructural 0.7040 puntos menor a lo largo de toda la pandemia (p-value = 0.0029). Nota metodológica: Esto resuelve la paradoja entre el modelo Pooled OLS y Efectos Fijos, demostrando que el desempleo crónico baja la movilidad general, pero los shocks temporales de desempleo la suben temporalmente. Media de Población (mean_Population): Las ciudades que son históricamente más pobladas tienen, en promedio, 0.0930 puntos porcentuales más de movilidad que las ciudades más pequeñas. Media de Sexo (mean_sex_ratio): Tener una proporción histórica promedio mayor reduce la movilidad estructural en 2.4411 puntos porcentuales.

Pregunta 7

Usando sus respuestas anteriores, que modelo prefiere? que se puede inferir en general respecto del efecto de las restricciones gubernamentales sobre la movilidad laboral?

A la hora de elegir el mejor modelo, la decisión se basa en las pruebas que fuimos construyendo paso a paso.

Primero, descartamos el modelo Pooled OLS porque es demasiado básico: asume que todas las ciudades se comportan igual e ignora por completo la identidad de cada una (cosa que el F-test nos demostró que era un error).

Luego comparamos Efectos Fijos (FE) y Efectos Aleatorios (RE) usando el Test de Hausman. El resultado fue súper claro (un p-value de 0.0000), lo que nos confirma empíricamente que las características ocultas de cada ciudad (su cultura de trabajo, su geografía, etc.) sí están fuertemente relacionadas con nuestras variables. Por ende, Efectos Fijos es matemáticamente superior a Efectos Aleatorios.

Sin embargo, el modelo que prefiero y considero más completo es el de Efectos Aleatorios Correlacionados (CRE). Elegir este modelo es la mejor decisión porque nos da lo mejor de ambos, logrando aislar el efecto puro a lo largo del tiempo sin sesgos (exactamente igual que Efectos Fijos), pero además nos permite ver el impacto de las variables que nunca cambian (como la industria o el país), algo que Efectos Fijos omite.

Tomando todo esto en cuenta, y basándonos en el modelo CRE, podemos inferir causalmente que las restricciones gubernamentales tienen un efecto directo y fuertemente negativo sobre la movilidad laboral. Al limpiar todo el ruido estadístico, vimos que cada vez que el gobierno aumenta en 1 punto su nivel de rigurosidad, la movilidad hacia los trabajos se contrae en 0.35 puntos porcentuales. Esto demuestra que las políticas públicas de encierro son efectivas.

Pregunta 8

Control Sintetico: Es posible que sus resultados anteriores tengan sesgo dado que la movilidad laboral y las restricciones son parte de un fenomeno dinamico en el tiempo. Utilice el notebook SynthControl.ipynb para estimar el efecto causal de las restricciones gubernamentales y la movilidad

para una ciudad de su eleccion, usando control sintetico (viendo las fechas semanales, deben elegir el periodo de tratamiento e indentificar los controles potenciales para la ciudad elegida, que no puede ser Zaragoza). Defina las variables para calcular el control sintetico y discuta sus resultados (instalar libreria pysyncon)

Análisis previo: Para este ejercicio, se seleccionó la ciudad de Francia con código FR008C que corresponde a NIZA. Francia tuvo fuertes variaciones en sus políticas de movilidad durante la pandemia.

```
[23]: import numpy as np
import pandas as pd
from pysyncon import Dataprep, Synth, AugSynth, PenalizedSynth
from pysyncon.utils import PlaceboTest
import matplotlib as mpl
import matplotlib.pyplot as plt
from IPython.display import display

[24]: # Filtra los datos solo para la ciudad elegida
francia_data = df[df['CODE'] == 'FR008C']

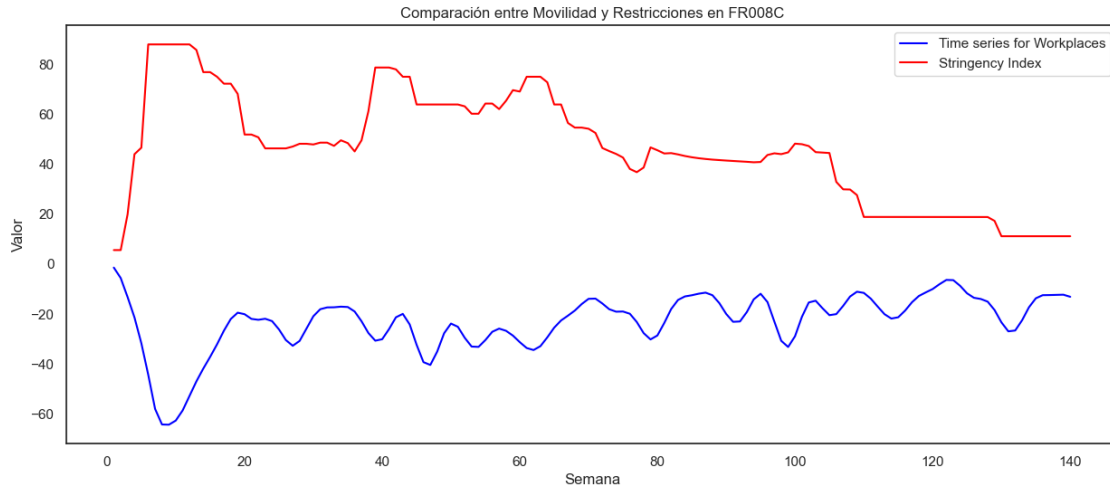
# Configura el tamaño del gráfico
plt.figure(figsize=(15, 6))

# Grafica la serie temporal 'workplaces' para FR008C
plt.plot(francia_data['week'], francia_data['workplaces'], label='Time series_
↳for Workplaces', color='blue')

# Grafica la serie temporal de las restricciones para FR008C
plt.plot(francia_data['week'], francia_data['Valor_Stringency_Index'],
↳label='Stringency Index', color='red')

# Configura etiquetas y leyenda
plt.xlabel('Semana')
plt.ylabel('Valor')
plt.title('Comparación entre Movilidad y Restricciones en FR008C')
plt.legend()

# Muestra el gráfico
plt.show()
```



```
[25]: df = pd.read_csv('dataset_prueba.csv')
city_data = df[df['CODE'] == 'FR008C'].sort_values('week')
pd.set_option('display.max_rows', None)
print("Historial de restricciones para FR008C:")
print(city_data[['week', 'Valor_Stringency_Index']].to_string(index=False))

pd.reset_option('display.max_rows')
```

Historial de restricciones para FR008C:

week	Valor_Stringency_Index
1	5.560
2	5.560
3	19.908
4	43.980
5	46.572
6	87.960
7	87.960
8	87.960
9	87.960
10	87.960
11	87.960
12	87.960
13	85.738
14	76.850
15	76.850
16	75.000
17	72.220
18	72.220
19	68.146
20	51.850
21	51.850

22	50.740
23	46.300
24	46.300
25	46.300
26	46.300
27	47.040
28	48.150
29	48.150
30	47.870
31	48.610
32	48.610
33	47.316
34	49.540
35	48.428
36	45.092
37	49.540
38	61.204
39	78.700
40	78.700
41	78.700
42	77.960
43	75.000
44	75.000
45	63.890
46	63.890
47	63.890
48	63.890
49	63.890
50	63.890
51	63.890
52	63.150
53	60.190
54	60.190
55	64.262
56	64.262
57	62.040
58	65.372
59	69.630
60	69.076
61	75.000
62	75.000
63	75.000
64	72.778
65	63.890
66	63.890
67	56.482
68	54.630
69	54.630

70	54.164
71	52.484
72	46.446
73	45.214
74	44.084
75	42.626
76	38.066
77	36.772
78	38.622
79	46.698
80	45.552
81	44.258
82	44.408
83	43.882
84	43.250
85	42.750
86	42.360
87	42.054
88	41.802
89	41.594
90	41.424
91	41.274
92	41.108
93	40.934
94	40.734
95	40.856
96	43.658
97	44.330
98	43.998
99	44.702
100	48.200
101	47.976
102	47.242
103	44.798
104	44.614
105	44.452
106	32.872
107	29.898
108	29.826
109	27.596
110	18.830
111	18.828
112	18.820
113	18.820
114	18.820
115	18.820
116	18.820
117	18.818

118	18.810
119	18.810
120	18.810
121	18.810
122	18.810
123	18.810
124	18.810
125	18.808
126	18.800
127	18.800
128	18.800
129	17.262
130	11.110
131	11.110
132	11.110
133	11.110
134	11.110
135	11.110
136	11.110
137	11.110
138	11.110
139	11.110
140	11.110

```
[26]: #Cargar los datos
df = pd.read_csv('dataset_prueba.csv')

#Filtrar solo las ciudades de Francia
df_fr = df[df['country'] == 'FR']

#Seleccionamos la ciudad FR008C y ordenamos cronológicamente por semana
city_data = df_fr[df_fr['CODE'] == 'FR008C'].sort_values('week')

#Calculamos la diferencia del índice de restricciones de una semana a otra
# Un valor negativo significa que las restricciones bajaron
diffs = city_data['Valor_Stringency_Index'].diff()

#Filtramos solo aquellas semanas donde hubo una caída drástica (mayor a 10)

caidas_drasticas = city_data[diffs < -10]

print("Semanas con caídas drásticas en las restricciones para FR008C:")
print(caidas_drasticas[['week', 'Valor_Stringency_Index']])
```

Semanas con caídas drásticas en las restricciones para FR008C:

	week	Valor_Stringency_Index
7719	20	51.850
7744	45	63.890

Al observar los datos, notamos que en la *semana 106* el índice de restricciones gubernamentales (Valor_Stringency_Index) cae de 44.45 a 32.8, marcando una fuerte liberación de las restricciones.

```
[27]: municipios_control = [municipio for municipio in df['CODE'].unique() if not
    ↪str(municipio).startswith("FR")]

dataprep = Dataprep(
    foo=df,
    predictors=[
        'Population',
        'agriculture',
        'industry',
        'construction',
        'age_dependency',
        'sex_ratio',
        'unemp',
    ],
    predictors_op="mean",
    time_predictors_prior=range(1, 106),
    special_predictors=[
        ('daily_cases', range(1, 106), 'mean'),
    ],
    dependent="workplaces",
    unit_variable="CODE",
    time_variable="week",
    treatment_identifier='FR008C',
    controls_identifier=municipios_control,
    time_optimize_ssr=range(1, 106),
)
```

```
[28]: pen = PenalizedSynth()
    pen.fit(dataprep, lambda_=0.01)
```

```
[29]: print(pen.weights().to_string())
    print(pen.weights())
```

```
DE001C    0.000
DE007C    0.000
DE025C    0.000
DE033C    0.000
DE081C    0.000
DE003C    0.000
DE014C    0.000
DE031C    0.000
DE012C    0.000
DE005C    0.000
DE019C    0.000
```

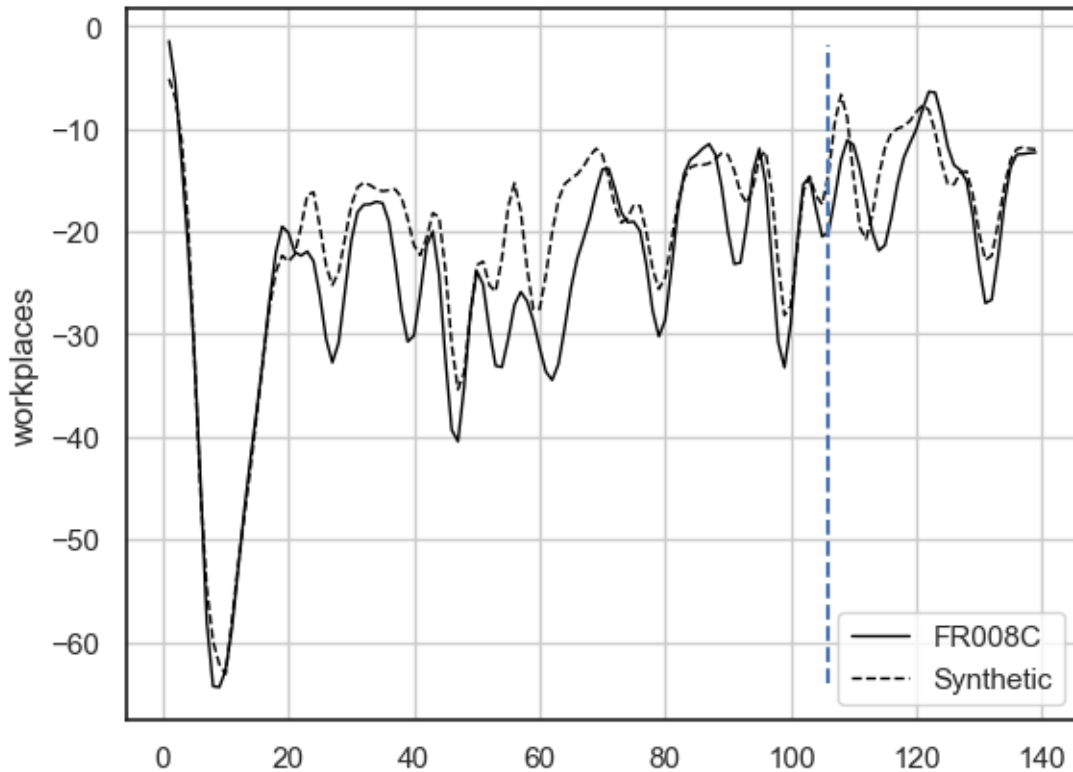
DE091C	0.000
DE002C	0.000
DE040C	0.000
DE013C	0.000
DE085C	0.000
DE004C	0.000
DE006C	0.000
DE010C	0.000
DE011C	0.000
DE015C	0.000
DE016C	0.000
DE032C	0.000
DE034C	0.000
DE080C	0.000
DE082C	0.000
DE083C	0.000
DE086C	0.000
DE087C	0.000
DE088C	0.000
DE124C	0.000
DE036C	0.000
DE089C	0.000
DE008C	0.000
DE009C	0.000
DE084C	0.000
DE017C	0.000
DE018C	0.000
DE030C	0.000
ES021C	0.000
ES060C	0.000
ES002C	0.000
ES019C	0.165
ES025C	0.422
ES062C	0.000
ES020C	0.000
ES008C	0.000
ES059C	0.000
ES010C	0.000
ES006C	0.000
ES022C	0.000
ES004C	0.000
ES003C	0.000
ES012C	0.000
ES005C	0.000
IT004C	0.000
IT002C	0.000
IT011C	0.000
IT012C	0.000

IT027C	0.000
IT006C	0.413
IT009C	0.000
IT007C	0.000
IT001C	0.000
IT003C	0.000
IT008C	0.000
IT005C	0.000
IT010C	0.000
IT064C	0.000
SE001C	0.000
SE003C	0.000
DE001C	0.0
DE007C	0.0
DE025C	0.0
DE033C	0.0
DE081C	0.0

	...
IT005C	0.0
IT010C	0.0
IT064C	0.0
SE001C	0.0
SE003C	0.0

Name: weights, Length: 71, dtype: float64

```
[30]: pen.path_plot(time_period=range(1, 140), treatment_time=106)
```

```
[31]: pen.summary()
```

```
[31]:
```

	treated	synthetic	sample mean
Population	462.846	468.677	673.403
agriculture	0.005	0.008	0.007
industry	0.064	0.104	0.117
construction	0.077	0.071	0.051
age_dependency	0.837	0.721	0.636
sex_ratio	113.725	112.659	105.812
unemp	20.000	15.345	11.338
special.1.daily_cases	4178.450	2800.678	2625.944

```
[32]: synth = Synth()
synth.fit(dataprep=dataprep, optim_method="Nelder-Mead", optim_initial="ols")
```

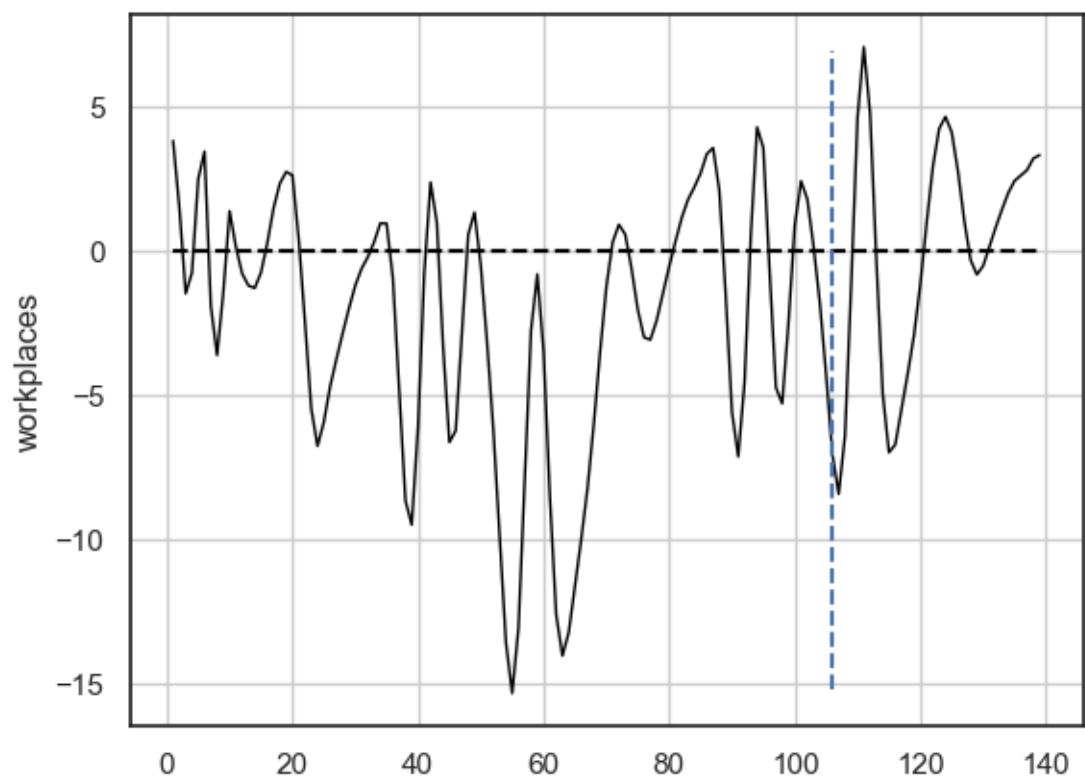
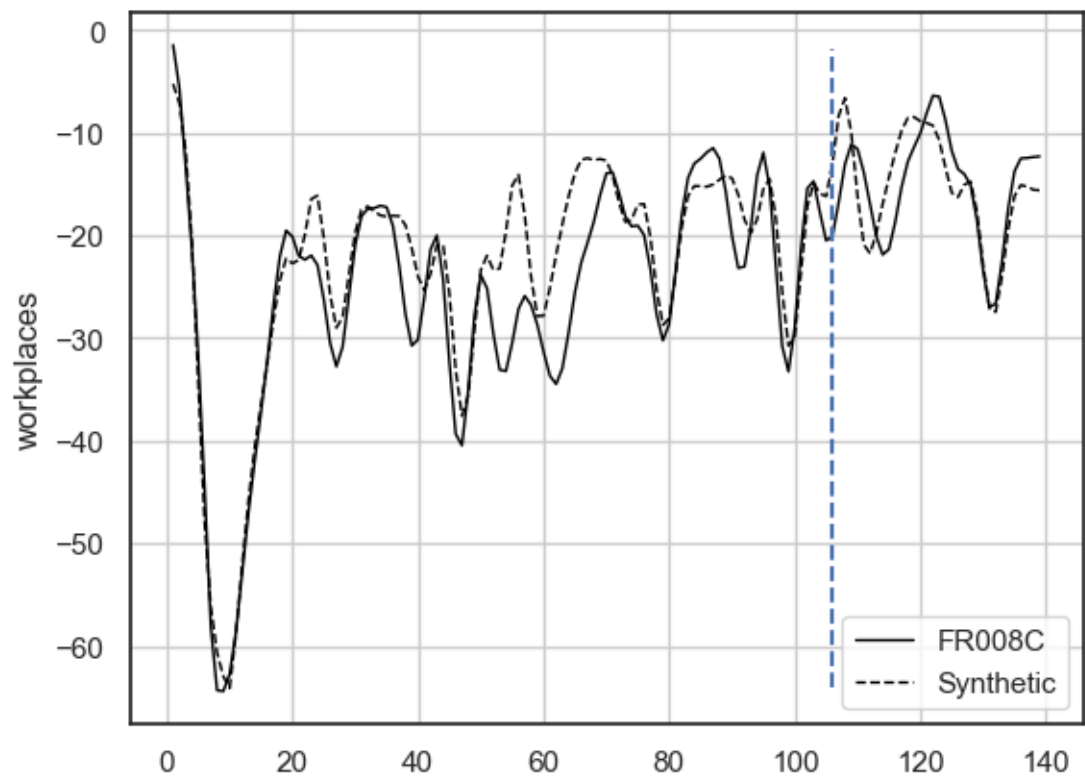
```
[33]: print(synth.weights().to_string())
print(synth.weights())
synth.path_plot(time_period=range(1, 140), treatment_time=106)
synth.gaps_plot(time_period=range(1, 140), treatment_time=106)
synth.summary()
```

```
DE001C    0.000
DE007C    0.000
```

DE025C	0.000
DE033C	0.000
DE081C	0.000
DE003C	0.000
DE014C	0.000
DE031C	0.000
DE012C	0.000
DE005C	0.000
DE019C	0.000
DE091C	0.000
DE002C	0.000
DE040C	0.000
DE013C	0.000
DE085C	0.000
DE004C	0.000
DE006C	0.000
DE010C	0.000
DE011C	0.000
DE015C	0.000
DE016C	0.000
DE032C	0.000
DE034C	0.000
DE080C	0.000
DE082C	0.000
DE083C	0.000
DE086C	0.000
DE087C	0.000
DE088C	0.000
DE124C	0.000
DE036C	0.000
DE089C	0.000
DE008C	0.000
DE009C	0.000
DE084C	0.000
DE017C	0.000
DE018C	0.000
DE030C	0.000
ES021C	0.000
ES060C	0.000
ES002C	0.000
ES019C	0.426
ES025C	0.000
ES062C	0.000
ES020C	0.000
ES008C	0.000
ES059C	0.000
ES010C	0.000
ES006C	0.000

ES022C	0.000
ES004C	0.000
ES003C	0.000
ES012C	0.000
ES005C	0.000
IT004C	0.000
IT002C	0.000
IT011C	0.000
IT012C	0.000
IT027C	0.000
IT006C	0.574
IT009C	0.000
IT007C	0.000
IT001C	0.000
IT003C	0.000
IT008C	0.000
IT005C	0.000
IT010C	0.000
IT064C	0.000
SE001C	0.000
SE003C	0.000
DE001C	0.0
DE007C	0.0
DE025C	0.0
DE033C	0.0
DE081C	0.0
	...
IT005C	0.0
IT010C	0.0
IT064C	0.0
SE001C	0.0
SE003C	0.0

Name: weights, Length: 71, dtype: float64



```
[33]:
```

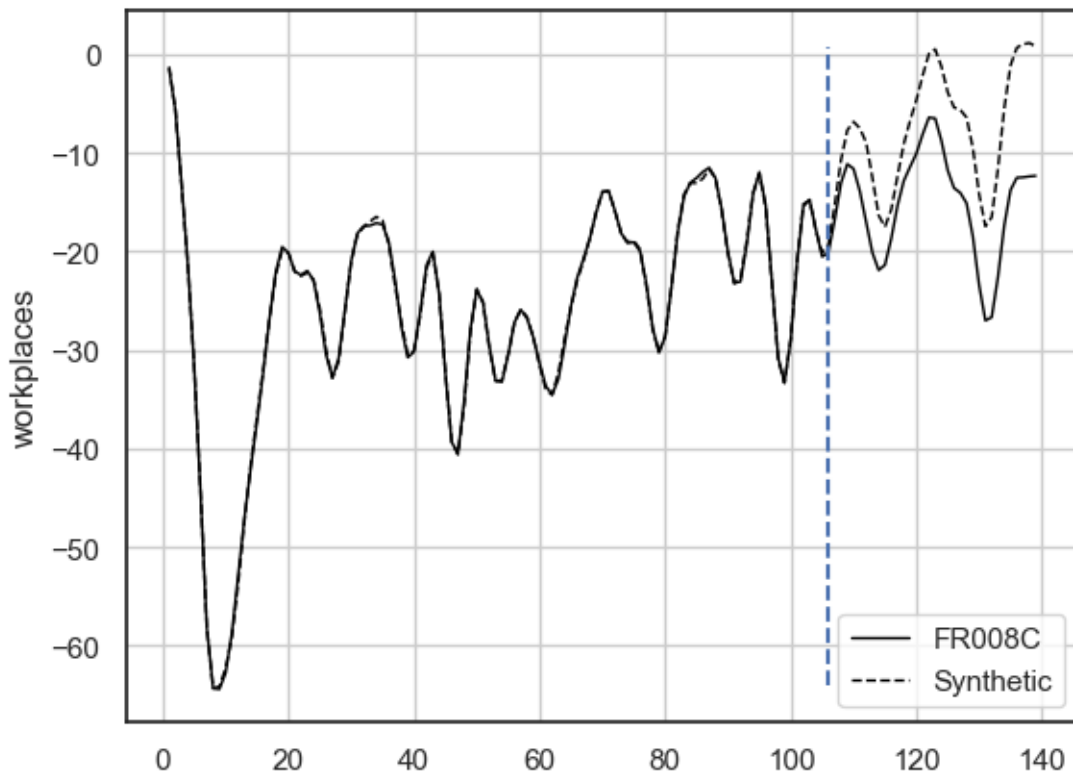
	V	treated	synthetic	sample mean
Population	0.208	462.846	663.105	673.403
agriculture	0.002	0.005	0.009	0.007
industry	0.036	0.064	0.139	0.117
construction	0.024	0.077	0.084	0.051
age_dependency	0.125	0.837	0.736	0.636
sex_ratio	0.008	113.725	110.249	105.812
unemp	0.466	20.000	18.234	11.338
special.1.daily_cases	0.132	4178.450	2800.934	2625.944

```
[34]: augsynth = AugSynth()
      augsynth.fit(dataprep=dataprep)
```

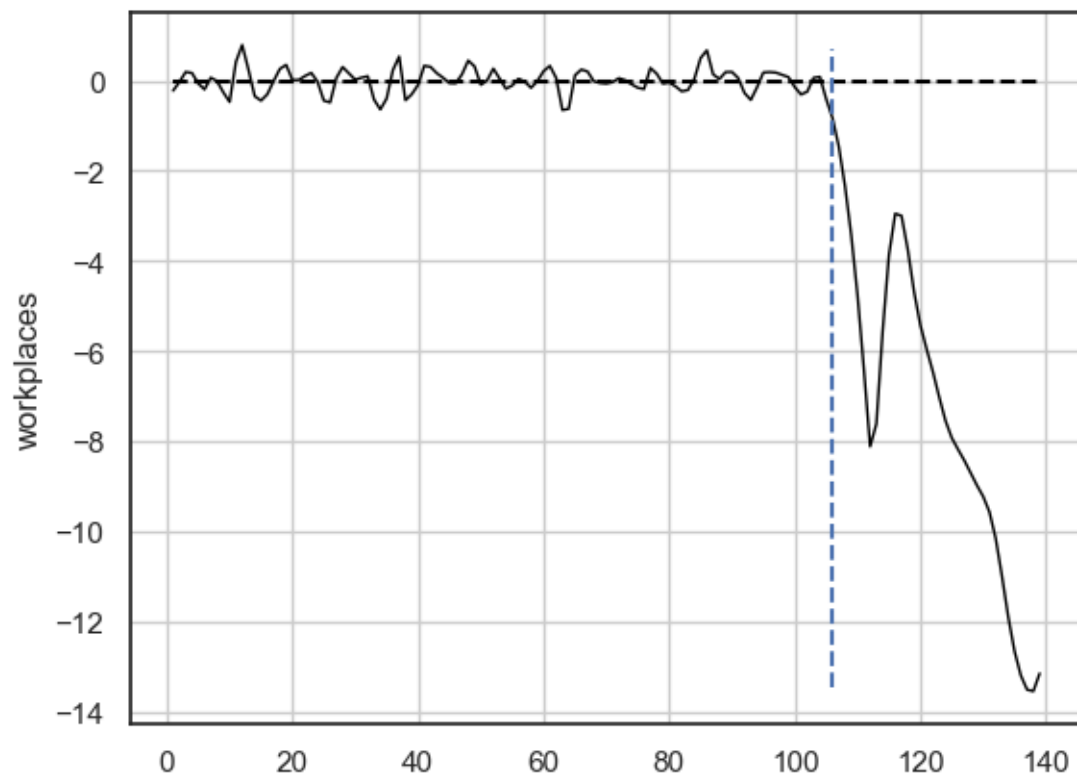
```
[35]: print(augsynth.weights())
```

```
DE001C    -1.537
DE007C    -0.121
DE025C     2.436
DE033C     0.294
DE081C     0.070
...
IT005C     0.649
IT010C     0.582
IT064C    -1.191
SE001C     0.419
SE003C    -1.213
Name: weights, Length: 71, dtype: float64
```

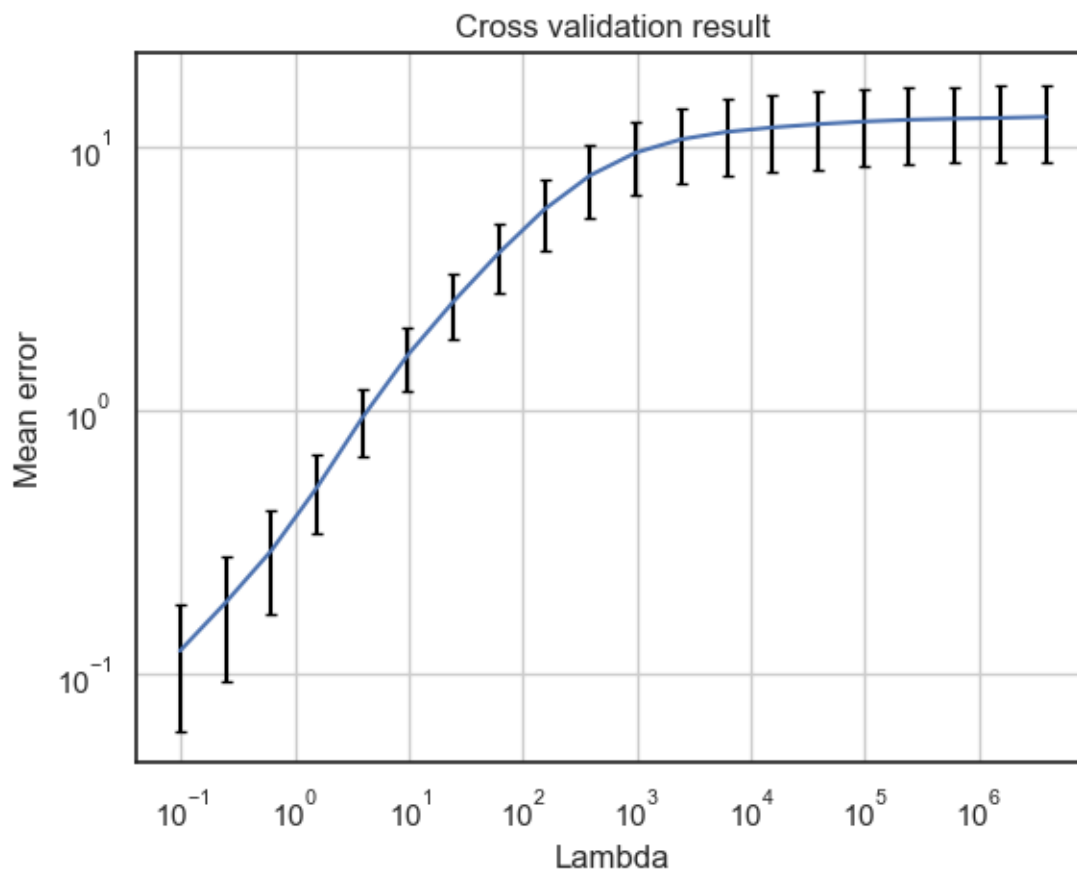
```
[36]: augsynth.path_plot(time_period=range(1, 140), treatment_time=106)
```



```
[37]: augsynth.gaps_plot(time_period=range(1, 140), treatment_time=106)
```



```
[38]: augsynth.cv_result.plot()
```



```
[39]: augsynth.summary()
```

```
[39]:
```

	treated	synthetic	sample mean
Population	462.846	463.457	673.403
agriculture	0.005	0.005	0.007
industry	0.064	0.064	0.117
construction	0.077	0.077	0.051
age_dependency	0.837	0.837	0.636
sex_ratio	113.725	113.721	105.812
unemp	20.000	19.904	11.338
special.1.daily_cases	4178.450	4169.611	2625.944

Descarte de los modelos Penalizado y Tradicional:

Al evaluar los métodos de Control Sintético Penalizado y Tradicional, notamos que no hacían un buen trabajo. Como se ve en las tablas de resumen (summary) y en los gráficos, estos modelos no lograron su tendencia de movilidad antes de la semana 106 de buena manera, así que fueron descartados.

Elección del modelo AugSynth (Aumentado):

Se decidió quedar con el control sintético aumentado, este modelo fue mucho más preciso, logró igualar casi perfectamente los datos de la ciudad de Niza como se ve en su gráfico, las dos líneas (real y sintética) van prácticamente juntas antes del tratamiento. Esto nos asegura que armamos un buen escenario de comparación.

Análisis del efecto real (Post-semana 106) Lo más interesante ocurre después de la semana 106. Uno esperaría que al quitar las restricciones, la gente volviera a la oficina. Sin embargo, el gráfico muestra lo contrario, la ciudad sintética recuperó su movilidad laboral mucho más rápido que nuestra ciudad real francesa.

Esto nos indica que, aunque el gobierno levantó las restricciones, la gente de esa ciudad no volvió a sus lugares de trabajo al ritmo esperado. Esto sugiere que levantar las medidas no fue suficiente para volver a la normalidad presencial, probablemente porque ya se habían consolidado nuevos hábitos a largo plazo en la ciudad, como el teletrabajo.

1.0.2 Anexo: Mapeo de Códigos y Ciudades

Código	Ciudad	País
DE086C	Aachen	DE
DE031C	Augsburg	DE
DE001C	Berlin	DE
DE016C	Bielefeld	DE
DE015C	Bochum	DE
DE032C	Bonn	DE
DE085C	Braunschweig	DE
DE012C	Bremen	DE
DE084C	Chemnitz	DE
DE010C	Dortmund	DE
DE009C	Dresden	DE
DE080C	Duisburg	DE
DE011C	Düsseldorf	DE
DE030C	Erfurt	DE
DE006C	Essen	DE
DE005C	Frankfurt am Main	DE
DE025C	Freiburg im Breisgau	DE
DE082C	Gelsenkirchen	DE
DE017C	Halle an der Saale	DE
DE002C	Hamburg	DE
DE013C	Hannover	DE
DE033C	Karlsruhe	DE
DE091C	Kassel	DE
DE036C	Kiel	DE
DE087C	Krefeld	DE
DE004C	Köln	DE
DE008C	Leipzig	DE
DE089C	Lübeck	DE
DE018C	Magdeburg	DE
DE081C	Mannheim	DE

Código	Ciudad	País
DE034C	Mönchengladbach	DE
DE003C	München	DE
DE083C	Münster	DE
DE014C	Nürnberg	DE
DE088C	Oberhausen	DE
DE040C	Rostock	DE
DE007C	Stuttgart	DE
DE019C	Wiesbaden	DE
DE124C	Wuppertal	DE
ES025C	A Coruña	ES
ES021C	Alicante/Alacant	ES
ES002C	Barcelona	ES
ES019C	Bilbao	ES
ES020C	Córdoba	ES
ES060C	Elche/Elx	ES
ES059C	Granada	ES
ES062C	Jerez de la Frontera	ES
ES008C	Las Palmas	ES
ES006C	Málaga	ES
ES010C	Palma de Mallorca	ES
ES004C	Sevilla	ES
ES003C	Valencia	ES
ES022C	Vigo	ES
ES012C	Vitoria/Gasteiz	ES
ES005C	Zaragoza	ES
FR024C	Angers	FR
FR005C	Bordeaux	FR
FR020C	Dijon	FR
FR026C	Lens	FR
FR003C	Lille	FR
FR004C	Marseille	FR
FR011C	Montpellier	FR
FR025C	Mulhouse	FR
FR007C	Nantes	FR
FR008C	Nice	FR
FR001C	Paris	FR
FR010C	Rouen	FR
FR009C	Strasbourg	FR
FR015C	Toulon	FR
FR013C	Tours	FR
IT008C	Bari	IT
IT009C	Bologna	IT
IT010C	Catania	IT
IT007C	Firenze	IT
IT006C	Genova	IT
IT064C	Messina	IT

Código	Ciudad	País
IT002C	Milano	IT
IT003C	Napoli	IT
IT027C	Padova	IT
IT005C	Palermo	IT
IT001C	Roma	IT
IT004C	Torino	IT
IT011C	Venezia	IT
IT012C	Verona	IT
SE003C	Malmö	SE
SE001C	Stockholm	SE